Frontier and Localhost: A Survey of How Production AI Learns Outside the Weights

Anonymous ACL submission

Abstract

Systems built on large language models increasingly adapt by editing the persistent text and code around a frozen model: instructions, skills, memories, tools, and routing. Such systems are often described as self-improving, a label that can cover three different things: an artifact that persists, an edit shown to improve the deployment that made it, and an improvement justified somewhere else. We separate them as two decisions. A local loop keeps an edit only when a check shows that the deployment's failure rate falls; a promotion loop decides how far a kept edit may travel. Reading 256 public systems by who or what makes each decision, we find that the local acceptance check is the most developed piece, while the failure log and the trace from a failure to the responsible artifact are often incomplete; promotion exists only in parts, inside one operator. The feedback a deployment receives (an exact check, an observed outcome, a judgment, or a delayed reward) fixes what a gate can know, and in every class an accepted edit is evidence only for the measure and tasks it was tested on. The reading exposes four confusions the label can hide. Every classification is released row by row.

1 Introduction

A coding agent keeps using the wrong naming convention in a repository. An engineer adds one line to its instructions, and the next session follows the convention. The model has not changed, but the deployed system has, as it does when an assistant corrects a stored preference, acquires a reusable skill, or learns which tool to call. We call this persistent layer the **scaffold**: instructions, skills, memory, tools, orchestration, and the evaluation and governance rules that control their use. In the language of compound AI systems (Zaharia et al., 2024), it is the part around the model that a deployment can inspect and revise without retraining. Much of the experience that makes a system useful in one setting now accumulates there, and systems that edit it are often described as self-improving.

That label can cover three different things. The first is an artifact that persists: a rule, a memory, or a skill that later sessions load. The second is an edit shown to improve the deployment that made it, which requires running the system again and comparing. The third is an improvement justified somewhere else, which requires evidence from the deployment that receives the edit rather than the one that wrote it. A rule added after one incident may prevent a recurring error, conflict with another rule, or merely lengthen the prompt; a corrected memory may remain in the history while the system goes on retrieving the outdated entry; a rule that helps one repository may be wrong in the next. Persistence settles none of this.

We separate the three as two decisions. The **local loop** keeps an edit only when a check shows that the deployment's failure rate falls. The **promotion loop** decides how far a kept edit may travel, to another project, team, or organization, with enough history to reverse that decision. We read 256 public research and production systems by who or what makes each decision: a program, a person, or nothing. The unit of analysis is the decision that accepts a persistent change, not the change.

The two loops are built unevenly. Nearly every system that adapts from failures decides local acceptance by program, while the steps that should feed that check, a log of failures and the trace from a failure to the artifact that caused it, are weaker. Promotion exists only in parts and only inside one operator: no surveyed system connects an automatic local gate to promotion across organizations with versioned lineage, access control, and rollback. The feedback a deployment receives, whether an exact check, an observed outcome, a judgment, or a reward that arrives months later, helps explain this pattern, because it fixes what an acceptance

gate can know, and in every class an accepted edit is evidence only for the measure and the tasks it was tested on.

The central contribution is this distinction between keeping, testing, and sharing an edit, and what it reveals about language agents that retain experience, test changes, and transfer learned artifacts. Read through it, the corpus shows four confusions that the self-improvement label can hide: detection mistaken for testing, history mistaken for correction, local success mistaken for transfer, and an evaluator that becomes part of what it evaluates. Around that distinction sit three supporting pieces: a classification of the corpus by who keeps an edit and who lets it travel, a four-class account of what feedback lets a gate know, and an audit of the four properties that governed sharing across organizations would require. The survey runs no experiments. Its classifications are one rater's reading of public evidence, released row by row in the supplementary material so that each can be checked.

2 Two decisions about an edit

Learning whether the naming rule works requires running the agent again on relevant tasks and comparing; the local loop makes that comparison routine (Fig. 1). It starts from a record of failures: rejected answers, user corrections, failed tool calls, evaluation regressions, unsuccessful tasks. Repeated failures are grouped into modes and prioritized by frequency and severity. For a selected mode, the system looks for the artifact responsible, since a stale memory entry calls for a different fix from a missing tool argument or an ambiguous instruction. Linking a failure to an editable part of the scaffold, a **coordinate**, is **credit assignment**. The system then proposes a candidate edit and tests it against the current scaffold. That test is the **gate**. An accepted edit is kept as a versioned artifact, and later evidence of regression can trigger rollback.

A deployment serves a particular distribution of users, tasks, and conventions, which we call its **patch**, and its **patch loss** measures failure on that distribution: a task-failure rate, a correction rate, a retrieval error. Write $L_D(\theta_s)$ for the patch loss of deployment D under scaffold θ_s, with the model weights fixed. No gradient runs through an instruction file, so the gate instead estimates the change in loss that a candidate edit z_t would cause and accepts the edit only when that estimate clears a margin:

$$G_t(z_t) = 1 \iff \widehat{\Delta}_{z_t} L_D(\theta_s^t) + \lambda \Omega(z_t) \leq -\tau_t,$$

where $\Omega \geq 0$ penalizes edit complexity and promotion breadth and $\tau_t \geq 0$ is the acceptance margin. Under further assumptions (Appendix B), improving edits eventually stop clearing the gate. This is **gate stability**, not global optimality: a limited proposer or a biased evaluator can leave useful changes undiscovered or rejected.

We call maintenance **artifact-layer descent** when five conditions hold: a measurable patch loss; a failure log that retains relevant failures; credit assignment that, at least sometimes, traces failures to the coordinates responsible; a candidate edit to those coordinates; and an acceptance gate that keeps or promotes an edit only after checking whether it lowers the loss. They separate it from patchwork, where an engineer adds a rule after an incident and measures nothing, and from random artifact search, which tests many variants and keeps a winner without tracing failures to their causes; descent uses the trace to direct the edit and tests the edit before keeping it. Where only some conditions hold, the optimization reading is limited accordingly.

The author of an edit and the authority that accepts it are separate: an agent may draft a revision that a person approves, and an engineer may write one that an automatic evaluation rejects. We classify each gate by who decides. **Auto** means a program applies a check, such as a verifier, an evaluation threshold, or a language-model judge (Zheng et al., 2023). **Human** means a person decides. **None** means no decision is made, because the loop is absent or every proposed edit is kept. A pair such as [Auto $\times$ Human] denotes a program at the local gate and a person at the promotion gate. The labels describe the decision mechanism; they do not by themselves establish that the decision is well supported.

Suppose the naming rule improves its repository and the team considers adding it to a shared template. That decision changes the population the rule acts on: other repositories may follow a different convention, and the local test says little about them. The promotion loop therefore asks for **receiving-side evaluation**, a check on the tasks of the deployment that would adopt the edit. The **evidence radius** is the scope for which the available evaluation supports a change, and a local success

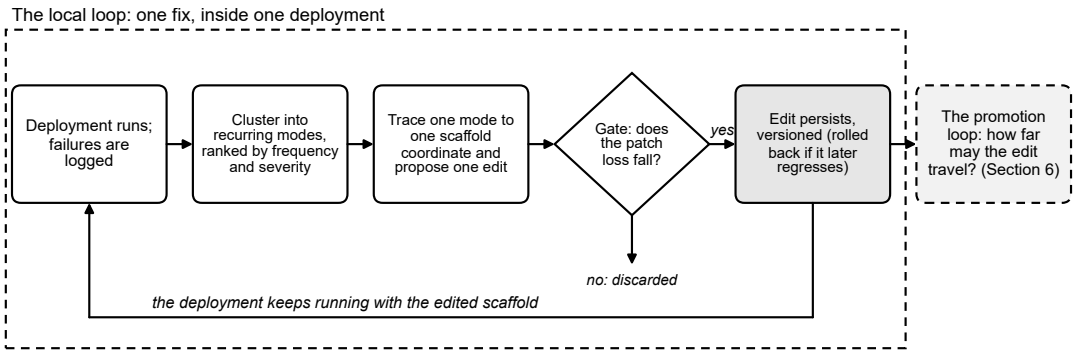


Figure 1: The local loop. Recurring failures guide a proposed scaffold edit; the local gate tests the change before it persists, and later regressions can trigger rollback. Sharing the kept edit is a separate decision, made by the promotion loop (Section 6).

does not enlarge it. Scopes nest from one person's account (USER) through a repository (PROJECT), a team's bundle of projects (STACK), and an organization (TENANT) to the scope all tenants share (CORE), as drawn in Fig. 4 in Appendix A.

The loss definition puts two limits on every such loop. An accepted edit is evidence of improvement only on the measure and the tasks evaluated, so promotion must preserve that meaning or establish the improvement again. And the loop optimizes a defined objective; it does not discover a better one by accumulating edits. Revising the evaluator changes the objective itself (Section 7).

3 Corpus and reading

We read each system's public method or documentation for four things: the persistent artifact it changes, the feedback that prompts the change, the decision that accepts it, and the scope in which the result is used. The corpus holds 256 systems from January 2023 to an October 2026 cutoff. A system enters if it adapts a foundation model through a persistent artifact updated from deployment signal, or if it is publicly described as self-improving, scaffolded, or agentic; the second group is a contrast set that tests whether those descriptions correspond to persistent adaptation.

Each system is scored from 0 to 5 on six substrates: instructions, skills, memory, tools, orchestration, and governance. A system that scores at least 3 on some substrate joins the core corpus of 204 systems; the other 52 form the contrast corpus. A second filter, the two-loop audit, asks whether session feedback changes a persistent artifact and whether that artifact is then promoted beyond the deployment that produced it, with versioning or lineage.

The audit admits 11 research systems (NanoResearch, SkillOpt, AlphaEvolve, ADAS, DGM, Voyager, SAGE, EvolveR, CoEvoSkills, AutoManual, and CASCADE) and 3 industry exemplars (Sierra Agent OS 2.0, Cognition Devin, and Letta's Context Repositories). Passing does not mean governed sharing. Every passer promotes within one operator, into a shared bank or archive or across the agents and sessions of one deployment, and Letta passes on the lineage of its shared-memory updates although they have no acceptance gate.

A single rater scored every system, rounding ambiguous cases down. Each row of the released table carries its source and a one-line justification. For the 126 most recently added systems, the evidence tier, gate types, feedback class, and governance properties are explicit columns as well, so counts over those columns apply to those systems only (Appendix A).

4 What the local loop closes

A system can detect a failure, explain it, and write a plausible fix without ever showing that the fix helps. SkillForge's judge agrees with people more than nine times in ten and picks the replies that should be rewritten. Those replies drive updates to the skill, and nothing tests the rewrite before it is committed (Liu et al., 2026b); the calibration concerns which replies are inadequate, not the effect of the resulting skill change. SkillRL separates the same steps by another route: a success-rate threshold triggers proposals for new skills, and the skill bank accepts each one unconditionally (Xia et al., 2026b). Both systems respond to experience, and both sit in [None × None]. Detection is not testing,

which is why the survey records who accepts an edit separately from who writes it or which signal triggers it.

Where a gate exists, it is the best-built step. Of the 126 systems added most recently, 105 are core, and 89 of those keep or reject edits by program. Acceptance also increasingly rests on a body of evidence rather than one favorable run, as in the paired per-item comparisons of SAGE-gate, a statistical acceptance method unrelated to SAGE (Wang et al., 2026c), and VaG's three heterogeneous critics that must all pass (Shang et al., 2026). Averaging independent noisy scores shrinks the variance of their mean, while requiring agreement among independent checks multiplies their false-acceptance probabilities, an idealized gain that real critics, which need not be independent, may not deliver (Appendix C).

The failure log that should feed the gate is rarer. Production memory more often records preferences, successes, or conversation history than failures, and only one field report in the corpus, from a single self-reported deployment, describes a store of dead ends kept to prevent their recurrence (Helwig, 2026). Where failures are retained, nearly every system uses them in arrival order; Task-CoEvolve's variance-weighted task sampling and AutoSaddler's mini-batch failure signals are exceptions (Miyai et al., 2026; Park et al., 2026).

Attribution is harder still. A failed output does not reveal whether its cause was an instruction, a retrieved memory, a skill, or the route through a group of agents; harness-optimization papers repeatedly name this the bottleneck (Lin et al., 2026a; Lee et al., 2026), and by downstream score alone an optimizer cannot be told from trial and error (Ong et al., 2026). Existing remedies, such as SkillsVote's attribution of an outcome to the skill, the exploration, or the environment before admitting an update (Liu et al., 2026a), usually work inside one substrate. None solves attribution across all six (Appendix G).

Who writes the candidate shapes the loop as much. Research systems usually generate it by reflecting on trajectories; in production prompt platforms an engineer writes it and session feedback never triggers one; and in five public skill repositories every substantive edit was committed under a named human account and merged by a person, most with an AI co-author trailer (Shen and Hruschka, 2026). Instruction files, the most widely deployed artifact (Appendix A), show what an unmeasured loop leaves open: one controlled study reports higher inference cost without improved task success (Gloaguen et al., 2026), and another reports lower median wall-clock time and fewer median output tokens, with total tokens about unchanged, without measuring correctness (Lulla et al., 2026b).

Maintenance also has to remove what no longer helps. Ratchet reports no gain from model-written skills over a no-skill baseline until lifecycle hygiene is applied (Zhang et al., 2026b), and eleven surveyed skill systems now treat removal as an operation that must pass a check (Zhang et al., 2026a; Wei et al., 2026; Pu et al., 2026; Wang et al., 2026d).

History is not correction either. Memory products have built one part of maintenance well, recording each entry's history: Letta's Context Repositories use git, Mem0 keeps per-memory history, and Zep represents facts bitemporally (Letta, 2026; Mem0, 2026; Zep, 2026). Yet a study of five production backends finds that all five still return facts that were revoked (Shen et al., 2026a). The record holds the change, but retrieval does not reliably enforce it.

Correction and reversal have their own limits. A governed collaborative memory that combines provenance, versioning, and correction earns the top memory score in the corpus (Cuadros et al., 2026). SkillSmith and SkillAudit support edit-level rollback (Wei et al., 2026; Gao et al., 2026a), but VaG finds that reverting a contaminated skill recovers little once other skills have been derived from it (Shang et al., 2026). The consequences of an edit can outlast the edit.

SkillOpt is a complete instance of all five conditions (Yang et al., 2026c). A frozen agent uses one markdown skill document; rollout and reflection batches supply the failure evidence, a consensus filter attributes recurring errors, bounded add, delete, and replace operations propose edits, and a candidate is kept only when its held-out exact-match score strictly improves. The follow-up SkillOpt-Lite drops a rejected-edit buffer and a slow update schedule, keeps the held-out gate and the consensus filter, and matches or exceeds the original (Shen et al., 2026b); in that setting, the gate and this form of credit assignment carry the weight (Appendix B).

5 What feedback lets a gate know

A coding agent changes a repository rule and receives a test result within seconds. A scheduling assistant changes its booking procedure and learns later whether an appointment was made. A marketing assistant changes its tone instructions and receives a reviewer's preference. A research assistant changes its strategy and may wait months to learn whether the direction was productive. All four can propose a persistent edit; what differs is the evidence available for accepting it. The signal's timing, exactness, and dependence on judgment determine how well a gate can estimate the change in loss, and Table 1 sorts the corpus into four classes on that basis, each assigned by the constraint that binds first (Appendix C).

Exact checks make an automatic gate easy to build, and they also show how narrow its acceptance can be. DGM keeps a self-modification when it passes a sandboxed compile-and-edit functionality check on ten benchmark tasks; the broader benchmark score weights which parent is selected from the archive and does not decide retention (Zhang et al., 2025a). Exactness belongs to the check, not to the deployment: an agent's outputs are stochastic, a finite suite samples only part of the task distribution, and passing the suite establishes only the properties it tests.

Observed outcomes are meaningful but noisy and confounded. A user can accept a wrong answer, and a merged pull request reflects more than the agent's technical work; Devin nonetheless reads merged pull requests as feedback (Cognition, 2025). Simulated outcomes, as in Voyager's game world or SAGE's AppWorld (Wang et al., 2025), remove the user confound and can be sampled again. In both cases an automatic gate needs a positive margin and enough episodes to absorb noise, and the result reveals that a task failed more readily than why.

Judged feedback has no independent answer to check against. For style, persuasion, design, and other preference-dependent tasks, the gate is a person, a rubric, or a model judge, and a judge's score counts as evidence of the preference only after it is calibrated against the people it stands in for. An uncalibrated judge is still an Auto gate, though its acceptances mean less. SkillForge, SkillChain, and SkillCoach report calibration measurements (Liu et al., 2026b; Hu et al., 2026b; Zhu et al., 2026b); no other surveyed judge does.

Delayed outcomes remove per-edit acceptance altogether. When a research direction is judged months later, after many intervening edits, the outcome cannot be assigned to each scaffold change. NanoResearch merges skills by semantic compatibility (Xu et al., 2026b) and CORAL promotes findings into shared memory on a heartbeat (Qu et al., 2026); these prune or consolidate proposals without estimating any edit's contribution to the eventual loss.

The classes yield three expectations. Local automatic acceptance should rest most directly on exact checks and observed outcomes; automatic promotion that estimates a loss change should appear only where a comparable check can run at the recipient, with people holding promotion for observed outcomes and judgments; and the dominant risks should differ by class, from suite overfitting and bloat under exact checks to drift under delay. The gate grid in Section 6 bears out the first two once one qualification is explicit: a program can merge two judged artifacts or prune a delayed proposal without estimating a loss change, so these are expectations about what a gate can establish, not about what software can execute. The failure surface in Appendix E sorts by class as the third expects, although no risk belongs to one class alone. In every class, an accepted edit is evidence only for the measure and the tasks it was tested on.

6 When an improvement is shared

Copying a rule file transfers the instruction, not the evidence that justified it. The recipient must either share the relevant evaluation conditions or test the change on its own workload, and a lower average loss at a broad scope can hide harm to one recipient. Local success is not transfer. A few systems already gather evidence beyond the source deployment, inside one operator. ShareMem consolidates one user's memories for another and evaluates them in the receiving context (Hu et al., 2026a); XRepoSkill verifies a rule in at least two repositories (Guo et al., 2026b); Evo2Team promotes routing skills between teams and confirms them at the target (Wang and Cui, 2026). Confirmation is necessary, not sufficient. Even in Evo2Team, nearly a fifth of confirmed transfers fail on graphs the confirmation did not cover.

Federated systems place the check elsewhere. EvolveNet merges a shared harness only when the merge fixes at least as many held-out items as it breaks, but the check runs at the aggregator (Nie

Class	The signal the gate receives	What the gate can be	What stays hard	Corpus examples
C1 exact	A check that returns pass or fail, or an exact score, at once and repeatably: tests, compilers, answer keys	An automatic check; one run per case, repeated only to sample the task mix	The lift is bounded and specific, an improvement in what the check measures on the cases it ran; the check names the failed output, not the coordinate that caused it; a check that is also the target can be gamed	DGM, AlphaEvolve, SkillOpt
C2 observable outcome	A real result one episode later: a booked slot, a merged pull request, a closed ticket; noisy and confounded	An automatic threshold over batched episodes; a person at promotion	The outcome says *that* the episode failed, not *why*; the channel can be poisoned	Devin, Sierra, SAGE, SkillRL, PACE, Voyager
C3 judged	A rater, a rubric, or a model judge; no ground truth	A judge calibrated against people; a person at promotion	The rubric is itself a scaffold coordinate, so the gate judges edits to itself; the proxy gets optimized past the point where it tracks quality	SkillForge, CASCADE, skills-repository review
C4 delayed	An outcome weeks or months later, once per many edits	No per-edit gate; proposals are pruned and milestones stand in	Credit across many edits; drift; the loop cannot be told from random search	NanoResearch, CORAL, the AI Scientist

Table 1: The four feedback-signal classes. Reading across a row: what the gate receives, what it can therefore be, and what it cannot fix.

et al., 2026); FederatedSkill's server agent applies, adapts, or rejects each client's edit (Yang et al., 2026a). Their clients are partitions or agent instances under one operator, not independently governed organizations, so neither instantiates a gate at a receiving organization.

Who lets an edit climb divides the corpus three ways. Research systems often promote by program, admitting a candidate to a bank or archive that later runs reuse; SkillOpt, DGM, ADAS, Voyager, SAGE, and AlphaEvolve decide this way (Hu et al., 2024; Wang et al., 2023; Novikov et al., 2025). That pool, inside one operator, is as far as any of them travels, with no access control, audited promotion, or gate at a receiving organization. Automatic admission also varies in what it means: NanoResearch's semantic merge, AutoManual's judged rewriting and consolidation, and ShareMem's judged consolidation are programmatic decisions, but none estimates a candidate's reduction in patch loss.

Production systems more often keep a person at the promotion gate. Sierra routes generated insights through human-authored expert answers (Sierra, 2025); in Devin, one pull-request merge both supplies the local outcome and records the reviewer's approval; and the research system CASCADE also combines human and agent approval for memory consolidation (Huang et al., 2025). The person stands in for the exact signal production lacks, which makes a program at the local gate and a person at the promotion gate the production-viable pairing rather than a compromise. Letta takes a third path, pushing shared-memory changes with git lineage and no acceptance check, while AutoAgent propagates nothing (Wang et al., 2026b). Both receive None at the promotion gate, one for sharing without checking and the other for not sharing.

Much of the sharing infrastructure exists apart from any learning loop. Coding assistants have converged on git-tracked markdown artifacts, with a tool layer organized around MCP-style interfaces (Galster et al., 2026), yet the runtime state of the adaptation process stays outside version control (Lulla et al., 2026a). MCP registries add namespaces and version records, governance registries add lineage and rollback, and prompt platforms such as Braintrust and LangSmith Hub add immutable versions, evaluation thresholds, and rollback (Braintrust, 2026; LangChain, 2026). Engineers author the candidates, and none of these systems generates one from failures. They move artifacts without carrying evidence that an artifact helps its recipient.

Fig. 2 places the two gates on one grid. [Auto × Auto] holds research systems that close both loops inside one environment or one operator; where their promotion gate estimates a loss change, the signal is an exact check or a simulated outcome. [Auto × Human] holds production systems with a re-

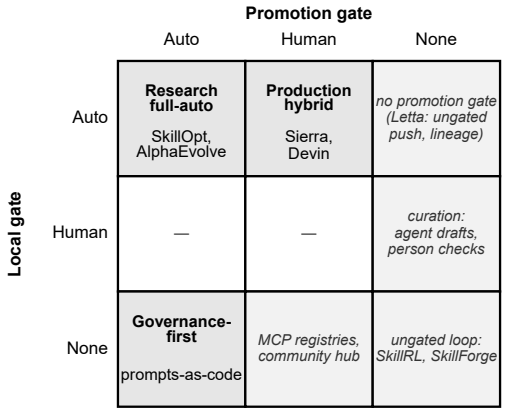


Figure 2: Local and promotion gates. Each axis records who accepts a change: a program (Auto), a person (Human), or no gate (None). None at the promotion gate includes both absent promotion and unchecked sharing; Section 6 distinguishes these cases, and Appendix D lists the occupants of each cell.

viewer on promotion, and CASCADE, with no exact-signal system. [Human × Auto] and [Human × Human] are empty: human local decisions occur only in curation, such as SkillChain's reflection gate, where a person decides which refined skills are deployed (Hu et al., 2026b), and curation promotes nothing. The bottom row holds prompt platforms, registries, and community hubs with no failure-driven loop, and [None × None] holds SkillRL and SkillForge. No relaxation of the audit yields governed promotion across organizations (Appendix D). No system has both the learning of the top row and the governance of the bottom.

7 When the evaluator changes

A pitch-review assistant consistently overrates decks that claim a large market without supporting numbers. The team revises its rubric from "evaluate market size" to "require a numeric estimate and a source." The edit may improve reviews, but it also changes the standard that will judge later behavior. If the same rubric judges future rubric revisions, the improvement process is modifying its own measuring instrument: the rubric is itself a scaffold coordinate, so the gate judges edits to itself. We call this the **reflexive gate**.

Calibration is the obvious defense, and it is missing exactly here: of the three scaffold systems that report calibrating a judge against people, none reports it for a judge that is itself being edited. The gap is keeping calibration while the criterion moves, which means evaluating the moving criterion as well as the outputs it scores. A fixed evaluator is not safe either. Optimizing a proxy beyond a point reduces the quality it was meant to track (Gao et al., 2022), and human evaluators' criteria change as they grade more outputs (Shankar et al., 2024). A gate needs its own held-out checks, calibration where judgment is involved, and a revision history.

Weight-level reinforcement learning meets the same problem when it extends verifiable rewards to open-ended tasks: some methods hold the judge fixed (Gunjal et al., 2025), others update the reward, rubric, or judge alongside the policy (Xu et al., 2026c; Ding et al., 2026a), and among the co-evolving methods reviewed only RLPR reports agreement with human labels, with a reward that is a probability rather than a rubric (Yu et al., 2025). The reflexive gate is one entry on a wider failure surface that includes scaffold bloat, poisoned memory, injection through tools, coordination failures, and drift. The literature, not this corpus, measured six of those failures (Appendix E).

8 Related work

Neighboring surveys organize the field by what, when, how, and where an agent evolves (Gao et al., 2026b), by inputs, agent system, environment, and optimizer (Fang et al., 2025), by the authority, scope, mutability, provenance, and recoverability of persistent state (Ding et al., 2026c), or by lifecycle phase and security objective for memory (Lin et al., 2026b). The last two are the closest governance precedents, and neither asks who gates an edit. None of the surveys classifies the gate that accepts an edit, the feedback that supports it, and the scope the edit reaches, and none covers promotion across organizations (Appendix H).

Prompt optimization supplies a second lineage. OPRO, DSPy, TextGrad, and their relatives optimize a prompt or program once against development evidence (Yang et al., 2024; Khattab et al., 2023; Yuksekgonul et al., 2024); they are not deployed loops. ACE persists an evolving playbook but never promotes it (Zhang et al., 2025b). These works establish many of the mechanisms read here. What they leave to this survey is the joint reading at deployment scope: whether an artifact persists, whether a local acceptance decision is made, which signal supports it, and what authorizes its use elsewhere.

A third body of work built the loop in pieces. Re-

flective evolution supplied proposal from trajectory evidence (Agrawal et al., 2025); reflective context learning supplied a generalization penalty, the Ω of the gate (Vassilyev et al., 2026); harness optimization supplied attribution within the orchestration graph (Lee et al., 2026; Nguyen et al., 2026); the lifecycle literature supplied gated retirement; and transfer and federation supplied the check on arrival. Each line inherited its loss from a benchmark and began from available trajectories, and building a record of recurring failures was none of their starting problems. This history helps explain why the local loop grew from evaluation backward toward the evidence needed to choose a useful edit (Appendix G).

9 Discussion and open problems

Why specialization should live outside the weights has a practical answer and a structural hypothesis. A deployment must follow its own conventions and avoid failures that are rare globally but frequent locally; per-customer adapters (Hu et al., 2021) can absorb some of this but bring separate training, evaluation, and serving lifecycles, and in practice they are rare. Adaptation lands in persistent artifacts, the one substrate that is cheap, inspectable, reversible, scoped to one customer, and updatable daily (Appendix I).

The structural hypothesis comes from two analyses of error. Arbuzov et al. (2025) argue that long-output reliability depends on a sparse set of key decisions rather than a uniform error probability at every token, an argument supported by long-range dependence measured on one natural-text corpus and by majority-voting gains on reasoning benchmarks, and limited when consequential decisions grow with the task, as in long-horizon agentic work. Arbuzov et al. (2026) argue that in a bounded deployment with recurring tasks, tools, and evaluator expectations, failures fall into a catalogue of modes that grows slowly with experience, so that under their assumptions the intervention budget grows polylogarithmically with sequence length and becomes constant once the catalogue saturates, while a finite catalogue cannot cover an unbounded domain. If these hypotheses generalize, a deployment may be improved through a manageable set of targeted interventions, persistent artifacts give those interventions an inspectable form, and on that hypothesis the gated, scoped artifact layer is not a workaround for an inadequate model but the architecture the error structure calls for. The survey does not test the hypotheses; its findings about gates, feedback, and promotion follow from the corpus independently of them.

Governed sharing across organizations is a consequence of these findings, not their motivation. It needs four properties at once: an auto-gated local loop, promotion between independently governed organizations, versioned lineage with access control, and rollback. FederatedSkill and EvolveNet come closest and stay within one operator. No surveyed system holds all four, and none rolls back a promotion across organizations, because none makes one. Where quality is judged, a human promotion gate is a design feature (Appendix F).

The open problems follow the decisions: attributing a failure to its editable cause across artifact types, where slot, intervention-based, and contrastive attribution offer narrower starting points (Nguyen et al., 2026; Chu et al., 2026; Zehle, 2026); evaluation at the receiving deployment, which Federated Agent Optimization proposes but does not implement (Yang et al., 2026b); keeping a judge calibrated while its rubric is edited; and evaluation under delayed reward, where an optimizer should be held to its claims about why an edit helps and not only to its final score. Only after those can governed sharing across organizations join the parts while preserving tenant isolation and accounting for artifacts derived from a change that is later withdrawn.

The frontier model generalizes. The local scaffold specializes. Reliability comes from governing that specialization: knowing what each change improved, where that evidence applies, and how the change can be reversed, including when the change is to the evaluator itself.

Limitations

Every classification in this survey is one rater's reading. A single rater scored all 256 systems, rounded ambiguous cases down, and used half-scores only when one criterion was clearly met and another partly met. No inter-rater agreement is reported, and independent re-rating remains future work. To make the reading checkable rather than merely asserted, each row of the released table carries its source and a one-line justification (and, for the 126 most recently added systems, its evidence tier), so a reader who disagrees with a score can locate the disagreement.

The evidence is public, and public evidence is uneven. It ranges from peer-reviewed studies and open-source artifacts to vendor claims. A production claim counts only when it names a benchmark or integration target, a blog or interview counts only when stronger evidence corroborates it, and production claims are not pooled with peer-reviewed evidence in corpus-level statistics. The classifications describe what systems document, not a complete inventory of private production behavior; a deployment that runs a governed loop without describing it would be missed.

The corpus has a cutoff and a selection rule. It covers January 2023 to October 2026, and systems released after the cutoff are not read. Including systems that are publicly described as self-improving, scaffolded, or agentic deliberately admits a contrast set of 52, so proportions over the whole corpus mix adapting systems with systems admitted for comparison. The corpus was also built in two passes, and gate types, feedback classes, and governance properties are explicit columns only for the 126 most recently added systems. Counts over those columns, such as the 89 of 105 core systems that close the local loop by program, describe those systems and are not gate counts over all 256.

The survey runs no experiments. Its findings are structural readings of published methods and documentation, and its feedback-class predictions are expectations checked against where systems sit, not causal tests. The error-structure hypothesis discussed in Section 9, that deployment failures concentrate in a slowly growing catalogue of recurring modes, is not tested here; the findings about gates, feedback, and promotion do not depend on it.

References

agentic-community. 2026. Mcp gateway & registry. https://github.com/agentic-community/mcp-gateway-registry.

AGENTS.md. 2025. AGENTS.md: A cross-vendor open standard for agent instructions.

Lakshya A. Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziems, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J. Ryan, Meng Jiang, Christopher Potts, Koushik Sen, Alexandros G. Dimakis, Ion Stoica, Dan Klein, Matei Zaharia, and Omar Khattab. 2025. GEPA: Reflective prompt evolution can outperform reinforcement learning. *arXiv preprint*.

Mikhail L. Arbuzov, Sisong Bei, Ziwei Dong, Dmitri Kalaev, and Alexey A. Shvets. 2025. Beyond exponential decay: Rethinking error accumulation in large language models. *arXiv preprint*.

Mikhail L. Arbuzov, Lee Mosbacker, Sisong Bei, Ziwei Dong, Dmitri Kalaev, and Alexey Shvets. 2026. The architecture of errors: From universal impossibility to patch-local LLM reliability. *arXiv preprint arXiv:2605.30628*.

Paul Barbaste, Tristan Darrigol, Germain Vu, and Tom Wiltberger. 2026. Harness engineering: Anatomy, architecture, and evolution of coding agents – a source-code study of eleven systems. *arXiv preprint arXiv:2609.00006*.

Braintrust. 2026. Use prompts in code. Braintrust documentation, https://www.braintrust.dev/docs/guides/prompts. Accessed 2026-10-03.

Qianshu Cai, Yonggang Zhang, Xianzhang Jia, Huajiang Zheng, Wei Xue, Jun Song, Xinmei Tian, and Yike Guo. 2026. MOSS: Self-evolution through source-level rewriting in autonomous agent systems. *arXiv preprint*.

Mert Cemri et al. 2025. Why do multi-agent LLM systems fail? *arXiv preprint*.

Check Point Research. 2025. MCPoison: Cursor IDE's MCP vulnerability (CVE-2025-54136). https://research.checkpoint.com/2025/cursor-vulnerability-mcpoison/.

Mingju Chen, Can Lv, Guibin Zhang, Heng Chang, and Shiji Zhou. 2026. Harnessforge: Joint harness and policy evolution for adaptive agent systems. *arXiv preprint arXiv:2606.01779*.

Zhaorun Chen, Zhen Xiang, Chaowei Xiao, Dawn Song, and Bo Li. 2024. AgentPoison: Red-teaming LLM agents via poisoning memory or knowledge bases. *arXiv preprint arXiv:2407.12784*.

Yuxin Cheng, Chang Liu, Hanxin Yu, Haochen Tan, Taiqiang Wu, Weiqiang Jin, Jie Ran, Kaibo Wang, Xiaoguang Li, Haoli Bai, Graziano Chesi, and Ngai Wong. 2026. Optimas: Automatically optimize multi-agent system. *arXiv preprint arXiv:2608.21918*.

Jaewon Chu, Jinwoo Seo, Jaewon Cho, Jeehye Na, Yunyang Xiong, Youngdae Kim, and Hyunwoo J. Kim. 2026. Agentgrad: Intervention-guided prompt optimization for multi agent systems. *arXiv preprint arXiv:2609.08572*.

Cognition. 2025. Devin's 2025 performance review: Learnings from 18 months of agents at work. https://cognition.com/blog/devin-annual-performance-review-2025.

Diego F. Cuadros, Abdoul-Aziz Maiga, Helen Meskhidze, and Andre Curtis-Trudel. 2026. Governed collaborative memory as artificial selection in LLM-based multi-agent systems. *arXiv preprint*.

Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. 2024. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. *arXiv preprint arXiv:2406.13352*.

Hongxin Ding, Baixiang Huang, Yue Fang, Weibin Liao, Zheng Li, Jinyang Zhang, Zhijing Wu, Junfeng Zhao, and Yasha Wang. 2026a. EvoRubrics: Dynamic rubrics as rewards via adversarial co-evolution for LLM reinforcement learning. *arXiv preprint arXiv:2606.23038*.

Kexin Ding, Yang Zhou, Can Jin, Feng Tong, Mu Zhou, and Dimitris N. Metaxas. 2026b. Agent skill evaluation and evolution: Frameworks and benchmarks. *arXiv preprint arXiv:2606.11435*.

Tianyu Ding, Aditya Nannapaneni, Bingfan Liu, and Ling Zhang. 2026c. Always-on agents: A survey of persistent memory, state, and governance in LLM agents. *arXiv preprint arXiv:2606.30306*.

Shibhansh Dohare et al. 2024. Loss of plasticity in deep continual learning. *Nature*, 632(8026):768–774.

Jinyuan Fang, Yanwen Peng, Xi Zhang, Yingxu Wang, Xinhao Yi, Guibin Zhang, Yi Xu, Bin Wu, Siwei Liu, Zihao Li, Zhaochun Ren, Nikos Aletras, Xi Wang, Han Zhou, and Zaiqiao Meng. 2025. A comprehensive survey of self-evolving AI agents: A new paradigm bridging foundation models and lifelong agentic systems. *arXiv preprint arXiv:2508.07407*.

Chrisantha Fernando, Dylan Banarse, Henryk Michalewski, Simon Osindero, and Tim Rocktäschel. 2023. Promptbreeder: Self-referential self-improvement via prompt evolution. *arXiv preprint arXiv:2309.16797*.

Matthias Galster, Seyedmoein Mohsenimofidi, Jai Lal Lulla, Muhammad Auwal Abubakar, Christoph Treude, and Sebastian Baltes. 2026. Harness engineering for agentic ai coding tools: An exploratory study. *arXiv preprint arXiv:2602.14690*.

Haowen Gao, Haoran Chen, Can Wang, Shasha Guo, Liang Pang, Zhaoyang Liu, Huawei Shen, and Xueqi Cheng. 2026a. Skillaudit: Ground-truth-free skill evolution via paired trajectory auditing. *arXiv preprint arXiv:2606.14239*.

Huan-ang Gao, Jiayi Geng, Wenyue Hua, Mengkang Hu, Xinzhe Juan, et al. 2026b. A survey of self-evolving agents: What, when, how, and where to evolve on the path to artificial super intelligence. *Transactions on Machine Learning Research*. ArXiv:2507.21046v4, 16 Jan 2026.

Leo Gao, John Schulman, and Jacob Hilton. 2022. Scaling laws for reward model overoptimization. *arXiv preprint arXiv:2210.10760*.

Thibaud Gloaguen, Niels Mündler-Sasahara, Mark Niklas Müller, Veselin Raychev, and Martin Vechev. 2026. Evaluating agents.md: Are repository-level context files helpful for coding agents? *arXiv preprint arXiv:2602.11988*.

Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. 2023. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. *arXiv preprint arXiv:2302.12173*.

Anisha Gunjal, Anthony Wang, Elaine Lau, Vaskar Nath, Yunzhong He, Bing Liu, and Sean Hendryx. 2025. Rubrics as rewards: Reinforcement learning beyond verifiable domains. *arXiv preprint arXiv:2507.17746*.

Jianyuan Guo, Zhiwei Hao, Chengcheng Wang, Cheng Fan, Tingzhang Luo, Hongguang Li, Ying Gao, Hefei Mei, Jiankun Peng, Rongjian Xu, Minjing Dong, Han Wu, Mengyu Zheng, Kai Han, Shiqi Wang, Chang Xu, and Yunhe Wang. 2026a. From question answering to task completion: A survey on agent system and harness design. *arXiv preprint arXiv:2606.20683*.

Qingyan Guo, Rui Wang, Junliang Guo, Bei Li, Kaitao Song, Xu Tan, Guoqing Liu, Jiang Bian, and Yujiu Yang. 2024. EvoPrompt: Connecting LLMs with evolutionary algorithms yields powerful prompt optimizers. In *International Conference on Learning Representations (ICLR)*. ArXiv:2309.08532.

Tao Guo, Song Guo, Junxiao Wang, and Wenchao Xu. 2022. PromptFL: Let federated participants cooperatively learn prompts instead of models – federated learning in age of foundation model. *arXiv preprint arXiv:2208.11625*.

Yaoqi Guo, Haoyang Zhou, Jiayi Zhang, Yiran Zhang, Yang Liu, Qiuyuan Chen, Qiang Lin, Hande Dong, Jie M. Zhang, and Zhenpeng Chen. 2026b. Xreposkill: Learning transferable skills for software engineering agents. *arXiv preprint arXiv:2609.36807*.

Prannay Hebbar, Yogendra Manawat, Samuel Verboomen, Alesia Ivanova, Selvam Palanimalai, Kunal Bhatia, and Vignesh Baskaran. 2026. Sia: Self improving ai with harness & weight updates. *arXiv preprint arXiv:2605.27276*.

Mike Helwig. 2026. Memory as infrastructure: Reliability engineering for persistent agent memory in months-long LLM-assisted development. *arXiv preprint arXiv:2609.05510*.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2021. LoRA: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*.

Jinming Hu, Haodong Zhao, Qi Jia, Die Chen, Tianhang Zhao, Sufeng Duan, and Gongshen Liu. 2026a. Learning from others, acting for you: Cross-user memory sharing for llm agents. *arXiv preprint arXiv:2609.32511*.

Shengran Hu, Cong Lu, and Jeff Clune. 2024. Automated design of agentic systems (ADAS). *arXiv preprint*.

Yimin Hu, Mengtao Xu, Hao Guo, Yuheng Song, Xiaoyong Zhu, and Bo Zheng. 2026b. Skillchain: Closing the loop on skill evolution for image-based e-commerce ai assistants. *arXiv preprint arXiv:2606.12984*.

Xu Huang et al. 2025. CASCADE: Cumulative agentic skill creation through autonomous development and evolution. *arXiv preprint*.

Daniel Jaroslawicz et al. 2025. How many instructions can LLMs follow at once? *arXiv preprint*.

Ruipeng Jia, Yunyi Yang, Yongbo Gai, Kai Luo, Shihao Huang, Jianhe Lin, Xiaoxi Jiang, and Guanjun Jiang. 2025. Writing-zero: Bridge the gap between non-verifiable tasks and verifiable rewards. *arXiv preprint arXiv:2506.00103*.

Zixin Jiang and Bing Dong. 2026. Buildrix: An open platform for sharing and benchmarking agentic ai skills in building engineering. *arXiv preprint arXiv:2606.25139*.

Xuefeng Jin, Jiashuo Zhang, Teng Cao, and Bin Yang. 2026. Beyond score prediction: LLM-based essay scoring and feedback generation via reinforcement learning with rubric rewards. *arXiv preprint arXiv:2607.19219*.

Peter Kairouz, H. Brendan McMahan, Brendan Avent, Aurélien Bellet, Mehdi Bennis, et al. 2019. Advances and open problems in federated learning. *arXiv preprint arXiv:1912.04977*. Published in Foundations and Trends in Machine Learning, vol. 14, no. 1–2, 2021.

Richard Kang and Vincent Wang. 2026. Registry-governed agent lifecycle: Completing EDDOps with evaluation-driven registration, promotion, and retirement on AWS AgentCore. *arXiv preprint arXiv:2607.00345*.

Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. 2023. DSPy: Compiling declarative language model calls into self-improving pipelines. *arXiv preprint arXiv:2310.03714*.

Robert Kirk, Ishita Mediratta, Christoforos Nalmpantis, et al. 2024. Understanding the effects of RLHF on LLM generalisation and diversity. In *ICLR 2024*.

LangChain. 2026. Prompt engineering concepts. LangSmith documentation, `https://docs.langchain.com/langsmith/prompt-engineering-concepts`. Accessed 2026-10-03.

Yoonho Lee, Roshen Nair, Qizheng Zhang, Kangwook Lee, Omar Khattab, and Chelsea Finn. 2026. Meta-Harness: End-to-end optimization of model harnesses. *arXiv preprint*.

Letta. 2026. MemFS; memory & dreaming; introducing context repositories: Git-based memory for coding agents. `https://docs.letta.com/letta-code/memfs`, `https://docs.letta.com/letta-agent/memory`, `https://www.letta.com/blog/context-repositories/`.

Gaotang Li, Bhavana Dalvi Mishra, Zifeng Wang, Jun Yan, Yanfei Chen, Chun-Liang Li, Long T. Le, Rujun Han, George Lee, Hanghang Tong, Chen-Yu Lee, and Tomas Pfister. 2026a. RubricEM: Meta-RL with rubric-guided policy decomposition beyond verifiable rewards. *arXiv preprint arXiv:2605.10899*.

Hanchen Li, Runyuan He, Qizheng Zhang, Changxiu Ji, Qiuyang Mang, Xiaokun Chen, Lakshya A. Agrawal, Wei-Liang Liao, Eric Yang, Alvin Cheung, James Zou, Kunle Olukotun, Ion Stoica, and Joseph E. Gonzalez. 2026b. Combee: Scaling prompt learning for self-improving language model agents. *arXiv preprint*.

Laizhen Li, Jiarui Li, Juanjuan Zhao, Kejiang Ye, Ye Li, Cheng-zhong Xu, and Xitong Gao. 2026c. Grow the harness, not the context: From strategy-free scaffolds to reusable specialist agents. *arXiv preprint arXiv:2609.26760*.

Jiahang Lin, Shichun Liu, Chengjun Pan, Lizhi Lin, Shihan Dou, Zhiheng Xi, Xuanjing Huang, Hang Yan, Zhenhua Han, Tao Gui, and Yu-Gang Jiang. 2026a. Agentic harness engineering: Observability-driven automatic evolution of coding-agent harnesses. *arXiv preprint*.

Zehao Lin, Xixuan Hao, Renyu Fu, Shaobo Cui, Kai Chen, Chunyu Li, Zhiyu Li, and Feiyu Xiong. 2026b. A survey on long-term memory security in LLM agents: Attacks, defenses, and governance across the memory lifecycle. In *Proceedings of EMNLP 2026*. ArXiv:2604.16548v3, 22 Sep 2026.

Chen Ling, Pei Chen, Albert Guan, Jiaming Qu, Shayan Ali Akbar, Madhu Gopinathan, and Erwin Cornejo. 2026. PACE: Two-timescale self-evolution for small language model agents. *arXiv preprint*.

Hongyi Liu, Haoyan Yang, Tao Jiang, Bo Tang, Feiyu Xiong, Yuyu Luo, and Zhiyu Li. 2026a. SkillsVote: Lifecycle governance of agent skills from collection, recommendation to evolution. *arXiv preprint arXiv:2605.18401*.

Nelson F. Liu et al. 2023. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics (TACL)*.

Xingyan Liu et al. 2026b. SkillForge: Forging domain-specific, self-evolving agent skills in cloud technical support. *arXiv preprint*. Accepted at ACM SIGIR 2026 Industry Track.

Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. 2024. The AI scientist: Towards fully automated open-ended scientific discovery. *arXiv preprint arXiv:2408.06292*.

Jai Lal Lulla, Vahram Nersesyan, Seyedmoein Mohsenimofidi, Christoph Treude, and Sebastian Baltes. 2026a. Loop engineering: Building blocks, adoption, and impact. *arXiv preprint arXiv:2608.21884*.

Jai Lal Lulla et al. 2026b. On the impact of AGENTS.md files on the efficiency of AI coding agents. *arXiv preprint*.

Yanki Margalit, Nurit Cohen-Inger, Erni Avram, Ran Taig, and Oded Margalit. 2026. Governed shared memory for multi-agent LLM systems. *arXiv preprint arXiv:2606.24535*.

H. Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Agüera y Arcas. 2017. Communication-efficient learning of deep networks from decentralized data. In *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS)*, volume 54 of *Proceedings of Machine Learning Research*, pages 1273–1282.

H. Brendan McMahan, Daniel Ramage, Kunal Talwar, and Li Zhang. 2018. Learning differentially private recurrent language models. In *International Conference on Learning Representations (ICLR)*. ArXiv:1710.06963.

Lingrui Mei, Jiayu Yao, Yuyao Ge, Yiwei Wang, Baolong Bi, Yujun Cai, Jiazhi Liu, Mingyu Li, Zhong-Zhi Li, Duzhen Zhang, Chenlin Zhou, Jiayi Mao, Tianze Xia, Jiafeng Guo, and Shenghua Liu. 2025. A survey of context engineering for large language models. *arXiv preprint arXiv:2507.13334*.

Mem0. 2026. Mem0 platform: Memory history, feedback, and changelog. `https://docs.mem0.ai/api-reference/memory/history-memory`, `https://docs.mem0.ai/api-reference/memory/feedback`, `https://docs.mem0.ai/changelog`.

Microsoft. 2026. Inventory and discover mcp servers in your api center. `https://learn.microsoft.com/en-us/azure/api-center/register-discover-mcp-server`.

Atsuyuki Miyai, Kiyoharu Aizawa, and Toshihiko Yamasaki. 2026. Task-coevolve: Efficient harness optimization via adaptive validation task selection. *arXiv preprint arXiv:2608.20169*.

Michael Nguyen, Wei Chen Tan, Nurul Aisyah Hassan, Arvind Raman, Li Hua Lim, and Ahmad Faiz Razak. 2026. Where does harness-optimization value live? Localized gains and the budget-splitting trap in self-evolving LLM agents. *arXiv preprint arXiv:2609.02889*.

Thong Q. Nguyen, Shubhang Desai, Raja Hasnain Anwar, Firoz Shaik, Vishwas Suryanarayanan, and Vishal Chowdhary. 2025. VerificAgent: Domain-specific memory verification for scalable oversight of aligned computer-use agents. *arXiv preprint arXiv:2506.02539*.

Jun Nie, Yonggang Zhang, Qianshu Cai, Yiu-ming Cheung, Xinmei Tian, and Bo Han. 2026. Evolvenet: Collaborative harness evolution for agent self-improvement. *arXiv preprint arXiv:2608.04968*.

Alexander Novikov et al. 2025. AlphaEvolve: A coding agent for scientific and algorithmic discovery. arXiv preprint arXiv:2506.13131. *Preprint*, arXiv:2506.13131.

Kai Tzu-iunn Ong, Minseok Kang, Dongwook Choi, Junhee Cho, Seungju Kim, Seungwon Lim, Geunha Jang, Minwoo Oh, Bogyung Jeong, Sunghwan Kim, Taeyoon Kwon, and Jinyoung Yeo. 2026. Towards direct evaluation of harness optimizers via priority ranking. *arXiv preprint*.

Vishakh Padmakumar and He He. 2023. Does writing with language models reduce content diversity? *arXiv preprint*.

Enrico Palumbo, Alexandre Tamborrino, Victor Ode, Ben Lacker, Adrià Casas Escoda, Jeremy Hopple, Marcus Better, James Leoni, Hugo Galvão, Hugues Bouchard, Mounia Lalmas, José Luis Redondo García, Abenezer Abebe, Ann Clifton, Anton Blomberg, Henrik Lindström, Dani Doro, and Christine Doig Cardet. 2026. Bootstrapping conversational recommendation agents at spotify: Synthetic data generation and self-improvement loops. *arXiv preprint arXiv:2609.30297*.

Sungho Park, Wonjoong Kim, Rongyuan Tan, Jue Zhang, Wook-Shin Han, Pengfei Gao, Chanyoung Park, Yongqiang Yao, Rao Fu, Elsie Nallipogu, Qingwei Lin, Saravan Rajmohan, and Dongmei Zhang. 2026. Autosaddler: Automatic harness optimization with durable updates from agent execution traces. *arXiv preprint arXiv:2608.23041*.

Hongji Pu, Xinyuan Song, and Liang Zhao. 2026. Skillops: Managing llm agent skill libraries as self-maintaining software ecosystems. *arXiv preprint arXiv:2605.13716*.

Xue Qin, Simin Luan, John See, Zeyd Boukhers, Cong Yang, and Zhijun Li. 2026. Governed capability evolution: Lifecycle-time compatibility checking and rollback for AI-component-based systems, with a proof-of-concept evaluation on embodied agents. *arXiv preprint arXiv:2604.08059*.

Ao Qu, Han Zheng, Zijian Zhou, Yihao Yan, Yihong Tang, Shao Yong Ong, Fenglu Hong, Kaichen Zhou, Chonghe Jiang, Minwei Kong, Jiacheng Zhu, Xuan Jiang, Sirui Li, Cathy Wu, Bryan Kian Hsiang Low, Jinhua Zhao, and Paul Pu Liang. 2026. CORAL: Towards autonomous multi-agent evolution for open-ended discovery. *arXiv preprint*.

Audrey Quessada-Vial. 2026. Agentic configuration management (ACM): A reference configuration model for governed agentic systems. *arXiv preprint arXiv:2608.11166*.

Romain Robbes, Théo Matricon, Thomas Degueule, Andre Hora, and Stefano Zacchiroli. 2026. Agentic much? adoption of coding agents on github. *arXiv preprint arXiv:2601.18341*.

Herbert Robbins and David Siegmund. 1971. A convergence theorem for non negative almost supermartingales and some applications. In *Optimizing Methods in Statistics*, pages 233–257. Academic Press.

Maxime Robeyns, Martin Szummer, and Laurence Aitchison. 2025. A self-improving coding agent. *arXiv preprint arXiv:2504.15228*.

Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M. Pawan Kumar, Emilien Dupont, Francisco J. R. Ruiz, Jordan S. Ellenberg, Pengming Wang, Omar Fawzi, Pushmeet Kohli, and Alhussein Fawzi. 2024. Mathematical discoveries from program search with large language models. *Nature*, 625:468–475.

Tanmay Sah, Dolly Sah, Harshul Jain, and Tanya Sah. 2026. Evoundo: Recoverability-constrained self-evolution for llm agent harnesses. *arXiv preprint arXiv:2608.28363*.

D. Sculley, Gary Holt, Daniel Golovin, Eugene Davydov, Todd Phillips, Dietmar Ebner, Vinay Chaudhary, Michael Young, Jean-François Crespo, and Dan Dennison. 2015. Hidden technical debt in machine learning systems. In *Advances in Neural Information Processing Systems 28 (NIPS 2015)*.

Yuval Shalev, Zifeng Ding, and Mateja Jamnik. 2026. Training language agents to learn from experience. *arXiv preprint*.

Linfang Shang, Ming Xu, Yiding Sun, Tianle Xia, Lingxiang Hu, Lan Xu, and Ning Zheng. 2026. When self-evolution backfires: Pre-commit gating against skill contamination in llm agents. *arXiv preprint arXiv:2608.05810*.

Shreya Shankar, J.D. Zamfirescu-Pereira, Björn Hartmann, Aditya G. Parameswaran, and Ian Arawjo. 2024. Who validates the validators? aligning LLM-assisted evaluation of LLM outputs with human preferences. *arXiv preprint arXiv:2404.12272*.

Chen Shen and Estevam Hruschka. 2026. Who maintains agent skills? A longitudinal study of human-governed, AI-assisted skill maintenance. *arXiv preprint arXiv:2609.05677*.

Yi Ting Shen, Kentaroh Toyoda, and Alex Leung. 2026a. Revoked but still authoritative: An empirical study of revocation enforcement in agent-memory systems. *arXiv preprint arXiv:2609.08258*.

Yifei Shen, Bo Li, and Xinjie Zhang. 2026b. SkillOpt-Lite: Better and faster agent self-evolution via one line of vibe. *arXiv preprint arXiv:2607.03451*.

Kejian Shi, Yixin Liu, Peifeng Wang, Alexander R. Fabbri, Shafiq Joty, and Arman Cohan. 2026. References improve LLM alignment in non-verifiable domains. *arXiv preprint arXiv:2602.16802*.

Sierra. 2025. Agent OS 2.0: From answers to memory and action.

Shivalika Singh et al. 2025. The leaderboard illusion. *arXiv preprint*.

Yongye Su, Wujiang Xu, Chaoji Zuo, and Elisa Bertino. 2026. ChronoMem: Version control and semantic rollback for large language model agent memory. *arXiv preprint arXiv:2607.27773*.

Theodore R. Sumers, Shunyu Yao, Karthik Narasimhan, and Thomas L. Griffiths. 2023. Cognitive architectures for language agents (CoALA). *Transactions on Machine Learning Research (TMLR)*.

Xiaolong Sun, Qichao Wang, Hangyu Li, and Liang Chen. 2026. Verifiable memory: Learning unified memory management with local and global verifiers for large language model agents. *arXiv preprint arXiv:2608.03137*.

Md Mehrab Tanjim, Jayakumar Subramanian, Xiang Chen, Branislav Kveton, Subhojyoti Mukherjee, Anlan Zhang, Sungchul Kim, Somdeb Sarkhel, and Sunav Choudhury. 2026. MOCHA: Multi-objective chebyshev annealing for agent skill optimization. *arXiv preprint*.

Rishabh Tiwari, Kusha Sareen, Lakshya A. Agrawal, Joseph E. Gonzalez, Matei Zaharia, Kurt Keutzer, Inderjit S. Dhillon, Rishabh Agarwal, and Devvrit Khatri. 2026. Learning, fast and slow: Towards LLMs that adapt continually. *arXiv preprint*.

TrueFoundry. 2026. Mcp gateway authentication and security. `https://www.truefoundry.com/docs/ai-gateway/mcp-gateway-auth-security`.

Nikita Vassilyev, William Berrios, Ruowang Zhang, Bo Han, Douwe Kiela, and Shikib Mehri. 2026. Reflective context learning: Studying the optimization primitives of context space. *arXiv preprint*.

Vellum. 2026. Release tags. Vellum documentation, `https://docs.vellum.ai/product/deployments/release-tags`. Accessed 2026-10-03.

Eric Wallace, Kai Xiao, Reimar Leike, Lilian Weng, Johannes Heidecke, and Alex Beutel. 2024. The instruction hierarchy: Training LLMs to prioritize privileged instructions. *arXiv preprint arXiv:2404.13208*.

Guanzhi Wang et al. 2023. Voyager: An open-ended embodied agent with large language models. *arXiv preprint*.

Haoyu Wang, Guangyuan Dong, He Liang, Zijing Zhang, Jiachen Luo, Chuang Liu, Chao Xue, and Hao Tang. 2026a. MemGuard: Persisting verifier signals for LLM-agent memory governance. *arXiv preprint arXiv:2608.21867*.

Jiongxiao Wang et al. 2025. Reinforcement learning for self-improving agent with skill library. *arXiv preprint*.

Renxiang Wang and Jiaming Cui. 2026. Evo2team: When do evolved skills transfer? from selection to deployment. *arXiv preprint arXiv:2609.34135*.

Xiaoxing Wang et al. 2026b. AutoAgent: Evolving cognition and elastic memory orchestration for adaptive agents. *arXiv preprint*.

Yihao Wang, Linhan Xia, Rui Liu, Zhaofeng Zhang, Hongyu Wu, Yang Yang, Jinglu He, Yu Guo, and Kai Lei. 2026c. Sage: A statistical acceptance gate for self-evolving agents. *arXiv preprint arXiv:2609.36043*.

Yixuan Wang, Yiyang Zhou, Yiming Liang, Congyu Zhang, Fuxiao Liu, Jiawei Zhou, and Huaxiu Yao. 2026d. Not all skills help: Measuring and repairing agent knowledge. *arXiv preprint arXiv:2606.15390*.

Ziming Wang. 2026. TOKI: A bitemporal operator algebra for contradiction resolution in LLM-agent persistent memory. *arXiv preprint arXiv:2606.06240*.

Yangbo Wei, Zhen Huang, Shaoqiang Lu, Junhong Qian, Qifan Wang, Chen Wu, and Lei He. 2026. Skillsmith: Co-evolving skills and tools for self-improving agent systems. *arXiv preprint arXiv:2606.01314*.

Xiyang Wu et al. 2026. Co-evolving LLM decision and skill bank agents for long-horizon tasks. *arXiv preprint*.

Boming Xia, Liming Zhu, Zhenchang Xing, Qinghua Lu, Dino Sejdinovic, and Xiwei Xu. 2026a. Harnessing agent skills: Architectural patterns and a reference architecture for skill-mediated LLM agents. *arXiv preprint arXiv:2606.20631*.

Peng Xia et al. 2026b. SkillRL: Evolving agents via recursive skill-augmented reinforcement learning. *arXiv preprint*.

Jinghan Xu, Yiyong Xiao, Wanru Shao, Hankai Liu, and Xinjin Li. 2026a. Memory provenance laundering in LLM agents: A non-amplification firewall for persistent memory. *arXiv preprint arXiv:2607.29167*.

Jinhang Xu et al. 2026b. NanoResearch: Co-evolving skills, memory, and policy for personalized research automation. *arXiv preprint*.

Ran Xu, Tianci Liu, Zihan Dong, Tony Yu, Ilgee Hong, Carl Yang, Linjun Zhang, Tao Zhao, and Haoyu Wang. 2026c. Alternating reinforcement learning for rubric-based reward modeling in non-verifiable LLM post-training. *arXiv preprint arXiv:2602.01511*.

Zhenlin Xu et al. 2026d. From storage to steering: Memory control flow attacks on LLM agents. *arXiv preprint*.

Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V. Le, Denny Zhou, and Xinyun Chen. 2024. Large language models as optimizers. In *International Conference on Learning Representations (ICLR)*. ArXiv:2309.03409.

Jingbo Yang, Guanyu Yao, Yang Zhang, Ramana Rao Kompella, Gaowen Liu, and Shiyu Chang. 2026a. FederatedSkill: Federated learning for agentic skill evolution. *arXiv preprint arXiv:2606.03143*.

Qiang Yang, Zhiqiang Kou, Xueyi Zhang, Dong-Dong Wu, Hanlin Gu, Jing Guo, Yang Liu, Di Jiang, and Qian Xu. 2026b. Federated agent optimization. *arXiv preprint arXiv:2610.01195*.

Yifan Yang, Ziyang Gong, Weiquan Huang, Qihao Yang, Ziwei Zhou, Zisu Huang, Yan Li, Xuemei Gao, Qi Dai, Bei Liu, Kai Qiu, Yuqing Yang, Dongdong Chen, Xue Yang, and Chong Luo. 2026c. SkillOpt: Executive strategy for self-evolving agent skills. arXiv preprint arXiv:2605.23904. *Preprint*, arXiv:2605.23904.

Caili Yu, Yiqi Wang, Jiaqi Zhang, Yiqun Duan, Mingkai Zheng, Zhangkai Wu, Kaize Shi, and Taotao Cai. 2026a. From faulty memories to corrected actions: Dependency-guided rollback repair for memory-augmented agents. *arXiv preprint arXiv:2608.10502*.

Tianyu Yu, Bo Ji, Shouli Wang, Shu Yao, Zefan Wang, Ganqu Cui, Lifan Yuan, Ning Ding, Yuan Yao, Zhiyuan Liu, Maosong Sun, and Tat-Seng Chua. 2025. RLPR: Extrapolating RLVR to general domains without verifiers. *arXiv preprint arXiv:2506.18254*.

Zhaochen Yu, Yingcheng Wu, Zhenfei Yin, Kaiyuan Chen, Zhe Zhao, Mengdi Wang, Shuicheng Yan, and Ling Yang. 2026b. Recursive experiential–working memory evolution for long-horizon agent harnesses. *arXiv preprint arXiv:2608.24876*.

Mert Yuksekgonul, Federico Bianchi, Joseph Boen, Sheng Liu, Zhi Huang, Carlos Guestrin, and James Zou. 2024. TextGrad: Automatic "differentiation" via text. *arXiv preprint*.

Matei Zaharia, Omar Khattab, Lingjiao Chen, Jared Quincy Davis, Heather Miller, Chris Potts, James Zou, Michael Carbin, Jonathan Frankle, Naveen Rao, and Ali Ghodsi. 2024. The shift from models to compound AI systems. Berkeley Artificial Intelligence Research (BAIR) Blog, https://bair.berkeley.edu/blog/2024/02/18/compound-ai-systems/.

Tom Zehle. 2026. CANTANTE: Optimizing agentic systems via contrastive credit attribution. *arXiv preprint*.

Eric Zelikman, Eliana Lorch, Lester Mackey, and Adam Tauman Kalai. 2024. Self-taught optimizer (STOP): Recursively self-improving code generation. In *First Conference on Language Modeling (COLM 2024)*. ArXiv:2310.02304.

Zep. 2026. Zep documentation: Facts; february 2026 deprecation wave; changelog. https://help.getzep.com/facts, https://help.getzep.com/february-2026-deprecation-wave, https://help.getzep.com/changelog/2026/9/24.

Jenny Zhang et al. 2025a. Darwin Gödel machine: Open-ended evolution of self-improving agents. *arXiv preprint*.

Jiayi Zhang, Jinyu Xiang, Zhaoyang Yu, Fengwei Teng, Xionghui Chen, Jiaqi Chen, Mingchen Zhuge, Xin Cheng, Sirui Hong, Jinlin Wang, Bingnan Zheng, Bang Liu, Yuyu Luo, and Chenglin Wu. 2024. AFlow: Automating agentic workflow generation. *arXiv preprint arXiv:2410.10762*.

Qizheng Zhang, Changran Hu, Shubhangi Upasani, et al. 2025b. Agentic context engineering: Evolving contexts for self-improving language models. arXiv preprint arXiv:2510.04618. *Preprint*, arXiv:2510.04618.

Xing Zhang, Yanwei Cui, Guanghui Wang, Ziyuan Li, Wei Qiu, Bing Zhu, and Peiyang He. 2026a. Library drift: Diagnosing and fixing a silent failure mode in self-evolving llm skill libraries. *arXiv preprint arXiv:2605.19576*.

Xing Zhang, Yanwei Cui, Guanghui Wang, Ziyuan Li, Wei Qiu, Bing Zhu, and Peiyang He. 2026b. Ratchet: A minimal hygiene recipe for self-evolving LLM agents. *arXiv preprint*.

Sen Zhao, Ruiqi Kong, Zuyu Zhang, Lifeng Shen, Xinyu He, Xu Zhang, and Qinghua Zhang. 2026. Topological coherence for self-evolving multi-agent systems. *arXiv preprint arXiv:2609.37953*.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems (NeurIPS 2023), Datasets and Benchmarks Track*. ArXiv:2306.05685.

Hao Zhou, Haichuan Hu, Tianyu Luo, Ye Shang, Chunrong Fang, Zhenyu Chen, Liang Xiao, and Quanjun Zhang. 2026. Self-evolving coding agents. *arXiv preprint arXiv:2608.03392*.

Wangchunshu Zhou, Yixin Ou, Shengwei Ding, Long Li, Jialong Wu, Tiannan Wang, Jiamin Chen, Shuai Wang, Xiaohua Xu, Ningyu Zhang, Huajun Chen, and Yuchen Eleanor Jiang. 2024. Symbolic learning enables self-evolving agents. *arXiv preprint arXiv:2406.18532*.

Han Zhu, Chengkun Cai, Yuanfeng Song, Xing Chen, Sirui Han, and Yike Guo. 2026a. Self-evolving deep research via joint generation and evaluation. *arXiv preprint arXiv:2606.04507*.

Jiayin Zhu, Kelong Mao, Yudong Guo, Dengbo He, Sulong Xu, Simiu Gu, and Yutao Yue. 2026b. Skillcoach: Self-evolving rubrics for evaluating and enhancing agentic skill-use. *arXiv preprint arXiv:2607.01874*.

Mingchen Zhuge, Wenyi Wang, Louis Kirsch, Francesco Faccio, Dmitrii Khizbullin, and Jürgen Schmidhuber. 2024. Language agents as optimizable graphs. In *Forty-first International Conference on Machine Learning (ICML 2024)*. ArXiv:2402.16823.

Wei Zou et al. 2024. PoisonedRAG: Knowledge corruption attacks to retrieval-augmented generation of large language models. *arXiv preprint*. Accepted at USENIX Security 2025.

A Corpus construction and scoring

Section 3 states the corpus and the reading in four paragraphs. This appendix gives what the body leaves out: the selection rule and the two filters, the systems that pass the two-loop audit, the promotion scopes, the substrate rubric with its distribution over the corpus, and the evidence rules behind every classification.

A.1 What is read for each system

We read each system's public method or documentation for four things: the persistent artifact it changes, the feedback that prompts the change, the decision that accepts it, and the scope in which the result is used. This allows systems with different names and application domains to be compared by their behavior.

A.2 Scope and selection

The corpus contains 256 unique systems from January 2023 through the October 2026 cutoff. The first search pass identified 130 systems by May 2026; the second added 126. Sources include research papers, public repositories, product documentation, and supported production reports; the search families, evidence tiers, and inclusion rules are recorded with the released data in the supplementary material.

A system enters the survey if it adapts a foundation model through a persistent artifact updated from deployment signal, or if it is publicly described as self-improving, scaffolded, or agentic. Including the latter group provides a contrast set: the survey can test whether those descriptions correspond to persistent adaptation. Systems whose only update mechanism is weight-level training are excluded, as are systems without public evidence of release or deployment.

We apply two filters. The first admits a system to the **core corpus** if it scores at least 3 on at least one scaffold substrate under the rubric below. Of the 256 systems, 204 form the core corpus and 52 the contrast corpus. A system may have one integrated row or several substrate-specific rows, so the 256 systems occupy 269 scored rows.

The second filter is a **two-loop audit**. It asks whether session feedback changes a persistent artifact and whether that artifact is subsequently promoted beyond the deployment that produced it, with versioning or lineage. Applied to the core systems and the additional paper-based classifications described in the supplementary material, it admits 11 research systems and 3 industry exemplars. Fig. 3 summarizes the selection.

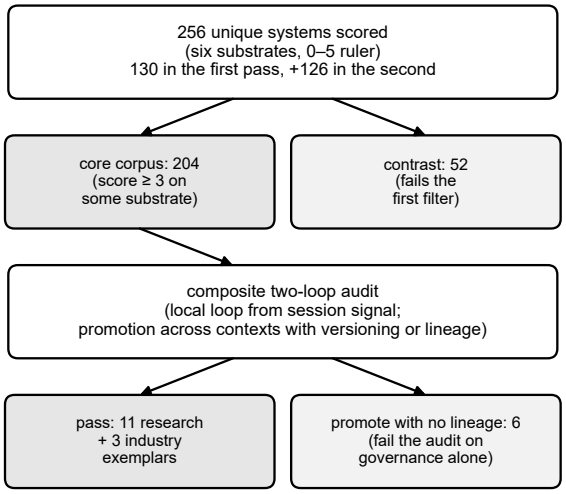


Figure 3: Corpus selection. The 256 systems occupy 269 scored rows. The first filter separates core from contrast systems; the two-loop audit then checks persistent local adaptation and promotion with lineage. The audit includes the additional paper-based classifications described in the text.

The research systems passing the audit are NanoResearch, SkillOpt, AlphaEvolve, ADAS, DGM, Voyager, SAGE, EvolveR, CoEvoSkills, AutoManual, and CASCADE. The industry exemplars are Sierra Agent OS 2.0, Cognition Devin, and Letta's Context Repositories. SkillOpt is classified from its paper and is not a row in the 256-system score table. The remaining paper-based systems are identified in the data notes, and the audit's boundary cases in the contrast set, both in the supplementary material.

Passing this audit does not establish governed sharing across organizations. All passing systems promote within one operator, into a shared bank or archive that later runs draw from, or across the agents and sessions of one deployment. Letta passes on the lineage of its shared-memory updates even though those updates have no acceptance gate. Six further systems promote across contexts without lineage on the promoted artifact; their treatment under relaxed audit rules is reported in Appendix D.

A.3 Promotion scopes

The scope in which an accepted edit is used is read on five nested levels. **USER** covers one person's account or fork. **PROJECT** covers a repository, workflow, or product workspace. **STACK** is

a team's curated bundle of projects. A **TENANT** is an organization or customer, and **CORE** is the scope shared by all tenants. A wider scope aggregates the losses of the deployments it contains. Promotion changes which population the edit is intended to improve, as shown in Fig. 4; Appendix F takes up what a check at the receiving scope can and cannot establish.

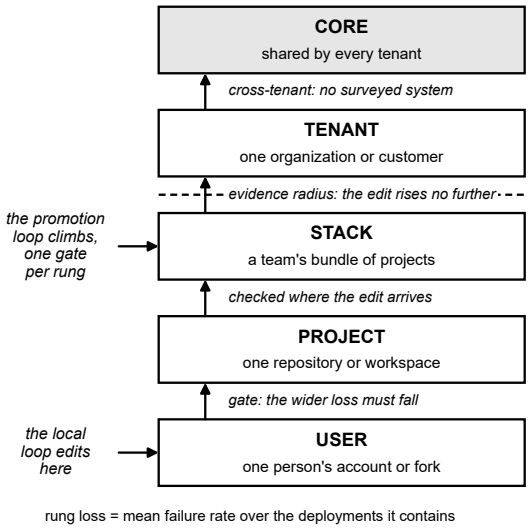


Figure 4: Promotion scopes, from a user account to the shared core. Wider adoption changes the population on which an edit acts. Receiving-side evaluation checks whether the improvement is supported for the recipient.

A.4 Substrates and scoring

The scaffold has six substrates: instructions, skills, memory, tools, orchestration, and governance. These are the possible targets of a proposed edit. Table 2 describes them; the episodic, semantic, and procedural memory categories follow Sumers et al. (2023).

The common scoring scale runs from 0 to 5. At 0 an artifact is ephemeral. At 1 it persists locally; at 2 it persists with tool attachment but without feedback. At 3 it either spans scopes without a gate or is updated from feedback by a person. At 4 a program updates it from feedback. The score-5 requirement depends on the substrate, as specified in the table.

This last distinction is important when interpreting the scores. Instructions and skills require both loops to reach 5. Orchestration requires a topology updated from deployment feedback by a loop outside the agent's own execution. Memory, tools, and governance are scored for their corresponding governance capabilities, so a registry can score 5 without generating local edits. Integrated rows measure breadth and depth of adaptation, not closure of either loop. The scale therefore supports substrate-specific comparisons; a top score is not a universal certificate of complete optimization.

Table 3 gives the corpus distribution. Research systems hold the top marks on skills, memory, and the integrated row. Production platforms and standards hold them on tools and governance. No system reaches the top score across all six substrates, and none reaches it on instructions or orchestration.

For the 126 most recently added systems, gate types, feedback classes, and governance properties are recorded as explicit columns of the released table, so counts based on those columns apply to those systems. For example, 89 of their 105 core systems close the local loop under a program. This is not a gate count over all 256 systems.

A.5 Instruction files

The instructions substrate never reaches the top score, yet it holds the most widely deployed artifact. Instruction files are present across Claude Code, Cursor, AGENTS.md (AGENTS.md, 2025), Windsurf, and Continue.dev, and adoption studies find them in roughly a fifth to a quarter of coding-agent projects (Robbes et al., 2026; Galster et al., 2026). Their prevalence does not settle their effectiveness. One controlled study reports higher inference cost without improved task success (Gloaguen et al., 2026). Another toggles the root AGENTS.md in paired runs on 124 pull requests and reports about 29% lower median wall-clock time and 17% fewer median output tokens, with total tokens approximately unchanged; it does not measure correctness (Lulla et al., 2026b). The studies measure different outcomes and disagree on cost. Neither establishes a general task-success benefit from accumulating instruction files.

A.6 Evidence and limitations

The evidence ranges from peer-reviewed studies and open-source artifacts to vendor claims. Every row of the released table carries its source and a one-line justification, and the rows for the 126 most recently added systems also carry an evidence tier. Peer-reviewed papers and open-source artifacts receive the greatest weight. A production claim must name a benchmark or integration target, and a blog or interview is counted only when stronger evidence corroborates it. Production claims are not

Substrate	What persists	A score of 5 requires
S1 Instructions	Project, team, or organization rules attached to a workspace	Versioned rules promoted under an automatic gate
S2 Skills	Procedures, as code or text, loaded on demand	A skill bank that edits itself on failure, test-gated, versioned, and promoted
S3 Memory	Experience kept across sessions: episodic, semantic, procedural	Provenance, versions, and a correction path for every entry
S4 Tools	The vetted action surface and its schemas	Evaluation-gated tool versions; a curated bundle as the product
S5 Orchestration	The multi-role graph and its routing	Topology updated from deployment feedback by a loop outside the agent's own run
S6 Governance	Versioning, promotion, audit, rollback	Staged promotion with evaluation thresholds, an audit log, and rollback

Table 2: The six scaffold substrates, what each holds, and what a system must do on it to score 5.

Substrate	n	Mean	Score-5 systems	What the corpus holds on it
S1 Instructions	15	3.07	(none)	Rules persist across IDEs and agents and nothing measures them; no system closes both loops; ceiling 4
S2 Skills	50	3.34	SAGE, COSPLAY	The local loop closes under a program in the research systems; the production repositories merge by hand
S3 Memory	50	3.06	Cuadros et al.	The ledger ships in products; the correction a gate would enforce is absent
S4 Tools	20	3.65	MCP, Harvey, Hippocratic AI	The most mature production substrate; registries govern admission, not evaluation
S5 Orchestration	37	3.08	(none)	Topologies change under an evaluation gate; one structurally gated instance; no two-loop system with lineage
S6 Governance	45	3.02	Braintrust, Vellum, LangSmith Hub, AGENTS.md under the Agentic AI Foundation (AAIF)	Prompts-as-code mature; registries add lineage and rollback; no local loop generates a candidate
Integrated	52	3.81	NanoResearch, AutoAgent, SkillRL	Breadth and depth of adaptation, not the loop; reached only by research systems without enterprise governance

Table 3: Substrate maturity across the 256-system corpus. Column n counts scored rows on each substrate, Mean is the average score, and Score-5 systems reach the substrate's top mark; the integrated row counts systems scored across several substrates as one row. Of the 126 most recently added systems, 105 are core and 32 are skill systems; 89 of those core systems close the local loop under a program, and none of the 126 scores 5 on any substrate.

pooled with peer-reviewed evidence in corpus-level statistics; vendor-based exemplars are identified as such.

One author scored all systems. Ambiguous cases were rounded down, and half-scores were used only when one criterion was clearly met and another partly met. Independent re-rating remains future work. The classifications should be read as an auditable interpretation of public evidence, not a complete inventory of private production behavior.

We also test the audit under alternative boundaries: counting rollback as sufficient promotion governance, allowing promotion without governance, and admitting loops that persist only optimizer state. Membership changes under these relaxations, but the principal structural findings remain: [Auto $\times$ Auto] stays well populated, the human-local cells hold only curation and registries, Letta alone passes with an ungated promotion, and no relaxation adds governed promotion across organizations (Appendix F). The sensitivity analysis is released with the data notes and the full score table with the released table, both in the supplementary material.

B Formal statement of the loop

Section 2 states the local loop in words and its acceptance rule in one line. This appendix carries the mathematics behind it: the patch loss and the gate, the update rule, a one-step descent inequality under a noisy gate, an informal convergence proposition, the optimization-analogy correspondence, four worked cases, and SkillOpt as a published instance of all five conditions. No new theorem is claimed; the proposition is the ordinary target of local stochastic optimization under noisy estimators, restated for a discrete, versioned parameter.

B.1 Patch loss and the acceptance rule

For deployment D, write the patch loss as

$$L_D(\theta_s) = \mathbb{E}_{x\sim D}[\ell(\pi(\theta_M, \theta_s, x))]\,,$$

where θ_M denotes the fixed model weights, θ_s the editable scaffold, and ℓ the operational loss. A proposal process uses the failure history to produce an edit z_t. There is no analytic gradient through an instruction file or memory entry. Instead, the gate estimates the change in loss and accepts when that estimate clears a required margin:

$$G_t(z_t) = 1 \iff \widehat{\Delta}_{z_t} L_D(\theta_s^t) + \lambda\Omega(z_t) \le -\tau_t.$$

Here $\Omega \ge 0$ penalizes edit complexity and promotion breadth, and $\tau_t \ge 0$ is the acceptance margin. An evaluation that blocks a regressing merge is a practical instance of this rule. A reviewer or governance process can also instantiate the gate by requiring stronger evidence for a broader or more complex change.

B.2 Setup

Let the frontier model have fixed parameters θ_M and let the scaffold be a structured object $\theta_s \in \Theta_s = \Theta_1 \times \cdots \times \Theta_6$ over the six substrate coordinates of Appendix A; Θ_s is discrete, symbolic, and versioned. For a deployment D, $L_D(\theta_s) = \mathbb{E}_{x\sim D}[\ell(\pi(\theta_M, \theta_s, x))]$. A candidate edit $z_t \sim Q_t(z \mid H_t, \theta_s^t)$ is drawn from a proposal process conditioned on the accumulated history H_t of failures, corrections, rejected outputs, evaluation regressions, and prior accepted edits. Q_t is not uniform: for a mode m that recurs with frequency f_m and carries per-occurrence loss s_m, its expected contribution to the failure rate is $f_m s_m$, so high-frequency or high-severity modes are proposed first. The finite directional difference is $\Delta_z L_D(\theta_s) = L_D(\theta_s \oplus z) - L_D(\theta_s)$; the gate sees only a noisy estimate $\widehat{\Delta}_z L_D$. The update is

$$\theta_s^{t+1} = \theta_s^t \oplus z_t \text{ if } G_t(z_t) = 1, \quad \theta_s^{t+1} = \theta_s^t \text{ otherwise.}$$

B.3 Descent inequality

Assume (A1) positive descent bias on accepted edits, $\mathbb{E}[\widehat{\Delta}_{z_t} L_D \mid G_t = 1, \theta_s^t] \le -\gamma_t - \tau_t$ for some $\gamma_t > 0$, and (A2) bounded estimator bias on accepted edits, $|\mathbb{E}[\Delta_{z_t} L_D - \widehat{\Delta}_{z_t} L_D \mid G_t = 1, \theta_s^t]| \le \varepsilon_t$. With $p_t = \Pr(G_t = 1 \mid \theta_s^t)$,

$$\mathbb{E}[L_D(\theta_s^{t+1}) \mid \theta_s^t] \le L_D(\theta_s^t) - p_t(\gamma_t + \tau_t) + p_t\,\varepsilon_t.$$

Per-step expected improvement is the gate's margin minus its estimator bias, weighted by acceptance probability. In the feedback-signal classes of Section 5, ε_t is near zero for the check itself in C1, with the sampling error of a finite suite over the patch remaining, bounded by batching for an observable outcome (C2), equal to the judge's bias and bounded only by calibration for a judged signal (C3), and unavailable per edit for a delayed signal (C4).

B.4 Convergence (informal)

Add (A3) proposal coverage with acceptance: at every non-gate-stable scaffold, Q_t places probability at least $q > 0$ on an admissible edit with $\mathbb{E}[\Delta_z L_D] \le -\delta$, and the gate accepts such an edit with probability at least $q' > 0$, so that $p_t(\gamma_t + \tau_t) \ge qq'\delta$ there; (A4) effective compactness of the reachable orbit; and (A5) summable bias, $\sum_t p_t\varepsilon_t < \infty$; and let $0 \le L_D \le B$. The one-step inequality is then of the Robbins–Siegmund form, a non-negative process that decreases in expectation up to a summable perturbation $p_t\varepsilon_t$, so $L_D(\theta_s^t)$ converges almost surely and $\sum_t p_t(\gamma_t + \tau_t) < \infty$ almost surely (Robbins and Siegmund, 1971). The finite sum carries the content: a bounded loss cannot pay for infinitely many accepted edits with a positive expected margin, so under (A3) the process visits non-gate-stable scaffolds only finitely often, and with (A4) the limit set of $\{\theta_s^t\}$ is gate-stable: no admissible edit clears the gate with negative expected directional loss in the limit. This is convergence of the loss and stability of the gate decision, not convergence to an optimum; a globally suboptimal gate-stable scaffold is consistent with the result, because Q_t may never propose the improving edit or $\widehat{\Delta}$ may be biased to reject it. In a non-stationary deployment ($D = D_t$) the same mechanism tracks a moving optimum rather than converging once, which is why recency weighting, pruning, expiry, and rollback are the scaffold-level analogues of continual-learning machinery rather than optional hygiene.

B.5 Correspondence

Table 4 pairs each optimization concept with its scaffold analogue. Credit assignment here is attribution, not backpropagation. Scaffold systems do not differentiate through instruction files. They trace an output error to an upstream coordinate, manually or with a judge or verifier. The descent argument needs only that this attribution succeeds

with non-zero probability per failure, which is the credit-assignment condition of Section 2.

B.6 Worked cases

Retrieval. Failures recur because retrieved context omits the decisive clause. Attribution traces the error to the retrieval coordinate: chunking, query rewriting, metadata filter, embedding model, reranker, or context-packing rule. The candidate edit tightens chunk boundaries or adds a metadata constraint, and the gate checks that recall and answer accuracy improve on the patch evaluation without degrading adjacent cases. Signal class C1 or C2.

Pitch review. The system repeatedly overrates decks that claim a large market without numbers, and the failure cluster points at an under-specified rubric coordinate. The edit changes "evaluate market size" into "require a numeric estimate and a source; if absent, cap the score"; the gate runs the revised rubric against held-out decks with human scores. What the edit changed was the gate that will judge the next edit: the reflexive gate of Section 7, class C3.

Tool use. Calls fail because optional parameters are under-specified. The responsible coordinate may be the tool schema, the call policy, or the clarification rule. The edit adds required fields or a pre-call uncertainty check; the gate checks that tool-call failures fall without raising refusals or latency. Class C2.

Memory. A stale customer preference is repeatedly retrieved and causes bad decisions. The responsible coordinate is a memory entry together with its provenance, priority, and expiry policy. The edit corrects, expires, or version-tags the entry; the gate checks that future behavior improves without erasing useful history. Class C2, with the correction itself observable only when a later outcome reveals it.

B.7 A complete local example

SkillOpt provides a concrete instance of all five conditions (Yang et al., 2026c). Its persistent state is one markdown skill document used by a frozen agent. Rollout and reflection batches supply the failure evidence. A consensus filter helps attribute recurring errors, and bounded add, delete, and replace operations propose changes. A held-out exact-match split supplies the loss, and a candidate is accepted only when its score strictly improves.

Across six benchmarks, seven target models, and three harnesses, SkillOpt is best or tied in every cell reported by its authors. Its follow-up, SkillOpt-Lite, removes a rejected-edit buffer and slow update schedule while retaining the held-out gate and consensus filter; it matches or exceeds the original (Shen et al., 2026b). This comparison supports the importance of the gate and this form of credit assignment in that setting, while identifying the removed controls as stabilizers rather than necessities.

SkillOpt also evaluates transfer across models, harnesses, and benchmarks. Those experiments measure how far the learned artifact remains useful. They are distinct from the acceptance decision that admits an edit to the shared skill bank.

C Gate families and feedback classes

Section 5 gives the four feedback classes with one developed example each (Table 1), and Section 4 states the difference between agreement and averaging in one sentence. This appendix lists the gate families that instantiate the acceptance test and the class each serves, sets out the batching distinction, and develops each class with the systems that fall in it.

C.1 Gate families

Section 2 collapses every production gate into one per-edit test G_t on an estimate of the loss change. Surveyed systems instantiate that test in several distinct ways, and some of them are better read as proposal pruning that precedes the per-edit gate than as estimators of $\widehat{\Delta}L_D$. Table 5 lists the families, the slot each occupies, representative systems, and the feedback-signal class each family serves.

The first five families populate G_t directly and, under the assumptions of Appendix B, inherit its descent guarantee. Semantic merge and archive selection are not estimators of $\widehat{\Delta}L_D$; they constrain the proposal distribution, and the descent argument applies to the composition of pruning and gate. Multi-sample agreement is listed because it is the correct analogue of "wait for k pieces of evidence before accepting an edit", a different operation from mini-batch averaging and one that should not be conflated with it. Read down the signal-class column. The only families available to a judged (C3) deployment are the two whose estimator is a judge. A delayed (C4) deployment has no family that estimates the loss change per

Optimization concept	Scaffold analogue
Parameter vector	Instructions, skills, memories, tools, routing graphs, and governance rules, evaluation gates among them
Loss	Failure, user correction, evaluation regression, retrieval miss, tool error, schema violation
Gradient estimate	Credit assignment identifying the coordinate responsible for a failure
Step	Accepted scaffold edit; step size is edit magnitude under $\oplus$
Aggregated update	Several recurring failures combined before an update, by averaging or by agreement across k proposals
Update radius	Promotion scope: which loss is descended (USER, PROJECT, STACK, TENANT, CORE)
Validation loss	Held-out patch evaluation before merge
Regularization	Complexity penalty $\Omega(z)$, bloat control, cross-patch regression checks
Overfitting	Scaffold improves on this deployment and degrades elsewhere
Rollback	Reverting a harmful accepted update

Table 4: Optimization concepts and their scaffold analogues.

edit at all: what it has is semantic merge before the gate and whatever proxy milestone stands at the horizon. That is the structural content of Section 5.

C.2 Agreement and averaging

An acceptance test can itself be too noisy to support a decision from one run. Several newer systems address this directly. SAGE-gate, a statistical acceptance method unrelated to SAGE, uses paired per-item comparisons before committing an edit (Wang et al., 2026c). VaG requires three heterogeneous critics to pass (Shang et al., 2026). AutoSaddler uses mini-batch failure signals in validation-based selection (Park et al., 2026).

These mechanisms should not be conflated. Averaging k independent noisy scores reduces the variance of their mean by a factor of $1/k$. Requiring agreement among k independent checks, each with false-acceptance probability p, gives the idealized probability p^k. The independence assumptions matter, and actual critics need not satisfy them. The shared development is that acceptance increasingly depends on a body of evidence rather than one favorable observation.

C.3 Assigning a class

The classes are assigned by the constraint that binds first. A long delay makes a signal C4 even when the eventual outcome is objective. Among timely signals, the absence of an independent ground truth makes the signal C3. Among timely objective signals, repeatable exact checks distinguish C1 from noisy observed outcomes in C2. A system may use several signals, but its operative acceptance decision must be read against the signal that supports it.

C.4 Exact checks (C1)

A compiler, proof checker, schema validator, or exact-match answer key can decide a specified property immediately and repeatably. This makes an automatic gate straightforward to implement. AlphaEvolve scores candidates through a cascade of exact evaluators (Novikov et al., 2025), and SkillOpt uses held-out exact-match evaluation (Yang et al., 2026c). DGM makes a narrower acceptance decision: it retains self-modifications that pass a sandboxed compile-and-edit functionality check on ten benchmark tasks. The broader benchmark score weights parent selection from the archive rather than determining retention (Zhang et al., 2025a).

Exactness applies to the check, not to the deployment as a whole. Repeating a deterministic check on the same output yields the same answer, but an agent produces stochastic outputs, and a finite suite samples only part of the task distribution. Estimating the change in patch loss still requires accounting for that variation. Passing the suite also establishes only the properties the suite tests.

Two problems remain even with a reliable checker. A failed test may not identify the instruction or skill that caused the failure, and repeated optimization against the same suite can overfit it. Cheap acceptance can also allow the artifact pool to grow without sufficient pruning. Exact feedback makes evaluation easier; it does not supply attribution or lifecycle management.

C.5 Observed outcomes (C2)

A booked appointment, closed ticket, merged pull request, or successful tool call provides an observable result after an episode. The result is meaningful but may be noisy or confounded. A user can accept an incorrect answer or reject a correct one, and a merged pull request reflects more than the

Gate family	What it estimates	Slot in Section 2	Signal class	Representative systems
Surrogate verifier	Pass or fail from tests, a symbolic checker, or an exact match	Per-edit gate G_t	C1	DGM (sandboxed compile-and-edit check; score weights selection, not retention), SkillOpt (held-out exact match), Voyager (game-state check: C2)
Evaluation cascade	$\widehat{\Delta}L_D$ on a held-out task suite, cheap-to-expensive evaluators	Per-edit gate G_t	C1 when the suite is exact; C2 when it is a task suite	AlphaEvolve, Braintrust, Vellum, LangSmith Hub
RL threshold	Reward improvement against a deployment threshold	Per-edit gate G_t	C2	SAGE, PACE
LLM judge with calibration	Pairwise or rubric score, calibrated against human agreement	Per-edit gate G_t	C3	SkillChain (judge stage, before its human reflection gate)
Reviewer judgment	Human credit assignment by pull-request review, expert sign-off, or a user's acceptance	Per-edit gate G_t, human-instantiated	C3 (and C2 when the reviewer has outcome data)	Sierra Agent OS 2.0, Anthropic skills repository, CASCADE (a person and an agent approve consolidation into shared memory)
Semantic merge	Whether a candidate duplicates or conflicts with existing artifacts	Proposal pruning before G_t	C4 (also a pre-filter in C2, C3)	NanoResearch orchestrator, FederatedSkill server agent, EvolveR (admission: cosine pre-filter plus LLM equivalence judgment on each distilled principle; retention: periodic pruning on per-principle success rate)
Archive selection	Whether a candidate improves on, or diversifies, an archive of already-scored programs	Proposal pruning before G_t	Any; inherits the class of the evaluator that scored the archive (C1 in AlphaEvolve)	AlphaEvolve (MAP-Elites-style database), ADAS
Multi-sample agreement	Whether k independent proposals or evaluations agree; false-accept rate decays as p^k rather than variance as $1/k$	Aggregation filter on the estimate	C2, C3 noise absorption	VaG (three heterogeneous critics must all pass); SkillOpt-Lite (consensus filter across rollouts)

Table 5: Gate families, their slot in the formalism, and the feedback-signal class each serves.

agent's technical performance.

Devin reads merged pull requests as feedback (Cognition, 2025). SAGE measures task success in AppWorld (Wang et al., 2025), and PACE uses failure signals from a deployed small model (Ling et al., 2026). SkillRL uses a success-rate threshold only to trigger proposals, without testing whether the proposed skills should remain (Xia et al., 2026b). The same kind of outcome can therefore support different operations in different systems.

Simulated outcomes, such as those in Voyager's game world or SAGE's AppWorld, remove the user confound and can be sampled repeatedly. They are treated as a distinguishable case within C2. For observed outcomes more generally, an automatic gate needs a positive margin and enough episodes or independent evidence to absorb noise. Even then, the result usually reveals that the task failed more readily than why it failed.

Promotion is harder because another deployment may differ in users, tasks, and acceptance behavior. A sender's successful outcome is not an evaluation of the recipient. This class also depends on channels that can carry poisoned memories or injected tool instructions, connecting the reliability of feedback to the integrity of its source.

C.6 Judgments (C3)

For style, persuasion, design, or other preference-dependent tasks, a gate relies on a person, rubric, reference comparison, or model judge. There is no independent exact answer to the preference being measured. CASCADE uses a human-and-agent gate for memory consolidation (Huang et al., 2025), and a reviewer approves promotion into Anthropic's skills repository. SkillForge uses its calibrated judge earlier in the process, to identify replies that motivate a rewrite (Liu et al., 2026b).

A program can make these decisions, but its score must be calibrated against the human judgment it is intended to represent before the score is treated as evidence of that preference. The gate labels deliberately do not record this distinction:

an uncalibrated model judge is still an Auto gate, though the meaning of its acceptances is less established. SkillForge, SkillChain, and SkillCoach report calibration measurements (Liu et al., 2026b; Hu et al., 2026b; Zhu et al., 2026b); no other surveyed judge does.

Optimization introduces another risk. A scaffold can become better at satisfying the judge while becoming worse at the underlying task. Reward-model research measures this form of proxy overoptimization (Gao et al., 2022). Human evaluation is not fixed either: Shankar et al. (2024) find that evaluators' criteria change as they grade more outputs. If the system also edits the rubric, it changes the instrument used to assess subsequent edits. This **reflexive gate** is developed in Section 7 and, with the weight-side comparison, in Appendix E.

C.7 Delayed and sparse outcomes (C4)

Research programs and open-ended discovery may produce meaningful outcomes only after weeks or months and many intervening edits. The AI Scientist illustrates the broad setting by generating ideas, conducting experiments, and writing a paper (Lu et al., 2024). The eventual outcome cannot usually be assigned to each preceding scaffold change.

Systems in this class consequently use intermediate decisions. NanoResearch merges skills by semantic compatibility (Xu et al., 2026b). CORAL promotes findings into shared memory on a heartbeat (Qu et al., 2026). These operations can prune or consolidate proposals, but they do not estimate each edit's contribution to the eventual loss. AlphaEvolve's archive diversity does not make it a delayed-signal example: its candidates have already been scored by exact evaluators, so archive selection is a search strategy within C1 (Novikov et al., 2025).

For C4, the descent interpretation applies only to whatever proxy gate is available or to an evaluation at the longer horizon. Temporal credit assignment remains unresolved, and the environment can change before the result arrives. Outcome gains alone are also insufficient to distinguish directed improvement from a successful search, a limitation emphasized in the evaluation of harness optimizers (Ong et al., 2026).

D The gate grid in detail

Section 6 reads the gate grid of Fig. 2 in one paragraph, and Section 5 states the three expectations the grid tests. This appendix gives the systems behind each local-gate and promotion-gate value, the infrastructure that fills the bottom row, the grid cell by cell with its boundary cases, and how the expectations fare against the corpus.

D.1 Who keeps a local edit

Research systems commonly generate candidates by reflecting on trajectories. In production prompt platforms, an engineer writes the candidate and session feedback never triggers one, while public skill repositories depend on contributors. A field study of five such repositories finds that every substantive edit was committed under a named human account and merged by a person, most carrying an AI co-author trailer (Shen and Hruschka, 2026). Where proposal generation depends on a person noticing a problem, the evaluation machinery can run automatically while the improvement process itself remains intermittent.

On skills, SAGE and COSPLAY close both local and promotion loops inside a research environment (Wang et al., 2025; Wu et al., 2026). SkillChain adds a human reflection decision after the judge, on which refined skills are deployed (Hu et al., 2026b). SkillRL and SkillForge leave the proposed changes unchecked. These differences would be obscured by classifying all three simply as systems that learn from feedback.

Orchestration has followed a similar progression. Early systems searched for agent topologies under an evaluation score (Zhuge et al., 2024; Zhang et al., 2024). More recent systems evaluate proposed topology changes before keeping them; TO-COMAS also requires a structural invariant (Chu et al., 2026; Chen et al., 2026; Zhao et al., 2026). OptiMAS is an exception: its optimizer reads trajectories and validation scores to write the next configuration, but the score informs the edit rather than supplying a keep-if-better decision (Cheng et al., 2026). No orchestration system closes both loops with lineage on the promoted artifact.

Human local gates appear only in curation. VerificAgent uses fact-checked, frozen memories (Nguyen et al., 2025), and in SkillChain a person decides, at a reflection gate, which refined skills are deployed. Prompt platforms and registries without a failure-driven proposal process have no local gate in this classification. Their checks act on release or admission to a shared pool, which belongs to promotion.

D.2 Who lets an edit climb

Research systems often promote by admitting a candidate to a bank or archive that later runs reuse. SkillOpt, DGM, ADAS, Voyager, and SAGE apply programmatic decisions within this shared environment (Yang et al., 2026c; Zhang et al., 2025a; Hu et al., 2024; Wang et al., 2023, 2025). AlphaEvolve and its predecessor FunSearch score candidates before archive admission (Novikov et al., 2025; Romera-Paredes et al., 2024). AlphaEvolve has produced gains used within Google, but it is a research loop within one operator, rather than a product for governed sharing between organizations. That pool is as far as any of them travels, and none has access control, audited promotion, or a gate at a receiving organization.

Not every automatic admission has the same evidentiary meaning. NanoResearch's semantic merge, AutoManual's judged rewriting and consolidation, and ShareMem's judged consolidation are programmatic decisions, but none of them estimates a candidate's reduction in patch loss. The distinction anticipated by the feedback classes is therefore essential: an automatic promotion decision can exist where a per-edit improvement estimate does not.

Production systems more often retain a person at the promotion decision. Sierra routes generated insights through human-authored expert answers (Sierra, 2025). In Devin, the same pull-request merge serves two roles: it supplies an observed outcome to local adaptation and records the reviewer's approval for the persistent repository (Cognition, 2025). Spotify's optimized prompts and plans enter production after online A/B testing and a launch decision (Palumbo et al., 2026). CASCADE also combines human and agent approval for memory consolidation (Huang et al., 2025). In each, the person at the promotion gate stands in for the exact signal production lacks, which makes a program on the local gate with a person on the promotion gate the production-viable pairing rather than a compromise.

Letta's Context Repositories take a different path. Shared-memory changes are pushed with git lineage but without an acceptance check. The record explains what was shared, while leaving whether it should have been shared unresolved. In many other systems, promotion is absent altogether. AutoAgent, for example, revises the tool and peer descriptions of one agent without propagating those revisions to another deployment (Wang et al., 2026b). Both cases receive None at the promotion gate, but for different reasons: one shares without checking; the other does not share.

D.3 What formats, registries, and release platforms provide

Much of the infrastructure needed for sharing already exists independently of these learning loops. Claude Agent Skills, Cursor Rules, GitHub Copilot instructions, Windsurf, Continue.dev, Zed, and Replit use git-tracked markdown artifacts attached to workspaces, with a tool layer organized around MCP-style interfaces (Galster et al., 2026; Barbaste et al., 2026). This convergence makes artifacts easier to carry between runtimes. A study of repository-level loops nevertheless finds that the runtime state of the adaptation process remains outside version control (Lulla et al., 2026a). The portable file does not necessarily carry the history that produced it.

Public repositories provide a shared scope with human admission. In a study of five skill repositories, every substantive change was committed under a named human account and merged by a person; only one of 254 edits deprecated a skill (Shen and Hruschka, 2026). MCP registries provide namespaces and version records, while control planes add scanning, access control, or quarantine (agentic-community, 2026; Microsoft, 2026; TrueFoundry, 2026). Governance registries add lineage and rollback (Kang and Wang, 2026; Quessada-Vial, 2026; Qin et al., 2026). EDDOps gates promotion on evaluation thresholds before a human sign-off, but none of these registries generates candidate artifacts from failures.

Braintrust, Vellum, and LangSmith Hub likewise provide immutable versions, evaluation thresholds, and rollback (Braintrust, 2026; Vellum, 2026; LangChain, 2026). Engineers author the candidates, and evaluations can block a regressing release. In the surveyed descriptions, session feedback does not trigger candidate generation or the release process. These platforms supply much of the machinery for controlled change without closing the failure-driven local loop.

D.4 The grid cell by cell

Table 6 summarizes the occupants of each cell of Fig. 2 and their signals. The grid distinguishes a system that generates and tests improvements from one that distributes human-authored artifacts under

admission control, while allowing their complementary capabilities to be compared.

The [Human × Auto] and [Human × Human] cells are empty in the surveyed set: no system combines a human local gate on a failure-driven loop with subsequent promotion. Human local decisions occur in curation. The bottom row includes comparison systems without a failure-driven local loop, such as prompt platforms, registries, and community hubs including Buildrix (Jiang and Dong, 2026). SkillRL and SkillForge occupy [None × None] because their failure-driven skill updates, new skills in SkillRL and rewrites in SkillForge, are kept unchecked.

The audit's boundary cases do not change this structure. Of six systems that promote without artifact lineage, five have sufficient evidence under the tier rule and enter already occupied cells when the audit is relaxed; Spotify is one of them. The sixth, Future AGI's prompt lifecycle, has only uncorroborated T5 evidence. No relaxation yields governed promotion across organizations. What the grid reveals is a division of capabilities: no system has both the learning of the top row and the governance of the bottom row.

D.5 The three expectations against the corpus

The feedback classification yields three expectations that can be checked against the corpus. First, local automatic acceptance is most directly supported by exact checks and observed outcomes. Judged acceptance depends on calibration, while delayed feedback cannot evaluate edits individually. Second, automatic promotion that estimates a loss change should occur only where a comparable check can be applied at the recipient, in exact and re-runnable simulated settings, and human promotion should hold the observed outcomes and judgments. Delayed feedback supports no per-edit gate on either half. Third, the main failure risks should differ: overfitting to a suite and bloat under exact feedback, corrupted outcome channels under C2, proxy optimization under C3, and drift under C4.

These are expectations about the evidence a gate can establish, not restrictions on whether software can execute a decision. A program can merge two judged artifacts or prune a delayed proposal without estimating a loss change. In every class, sharing remains an additional empirical question: even the same exact checker must be applied to the recipient's relevant cases before broader performance is established.

Read against the grid, the patterns match these expectations once the meaning of automatic admission is made explicit. Among [Auto × Auto] systems whose promotion gate estimates a loss change, the operative signals are exact checks or simulated outcomes. The judged and delayed cases use pruning, rewriting, or merging rather than that estimate. [Auto × Human] contains observed outcomes and judgments, with no exact-signal system in the surveyed cell. The third expectation is taken up with the failure surface in Appendix E.

E How adaptation can undermine its own evidence

Section 7 develops one failure, the evaluator that becomes part of what it evaluates, and Section 4 notes that rollback leaves derived artifacts behind. This appendix gives the whole failure surface: the seven failures and the loop piece that answers each, then accumulation and contamination, memory history that retrieval does not enforce, the changing evaluator with the weight-side comparison, and the failures of scope, coordination, and drift.

An improvement loop changes the context in which its next decision is made. New rules can conflict with old ones, a corrected memory can still be retrieved in its earlier form, and a frequently reused evaluator can become a target. These failures extend the technical-debt concerns identified for machine-learning systems by Sculley et al. (2015). The literature documents six of these failures in a measured case; a seventh, overfitting to one deployment, follows from the scope of the loss rather than from a measurement. Table 7 connects each to the control that addresses it.

E.1 Accumulation and contamination

As instructions accumulate, following them becomes harder and latency increases (Jaroslawicz et al., 2025). Ratchet makes the consequence visible at the system level: it reports no gain from model-written skills over a no-skill baseline until lifecycle hygiene is applied, while human-curated skills provide a double-digit gain (Zhang et al., 2026b). Conflicting or obsolete artifacts also complicate attribution: an evaluation may detect failure without revealing whether the cause is the new candidate or an older artifact. Gated retirement, pruning, and expiry preserve the conditions under which a new edit can be meaningfully assessed.

Cell	Who keeps the local edit, who lets it climb	Who is there	Signal found there
[Auto × Auto]	A program keeps it; a program lets it climb	Research systems that close both halves inside one environment or one operator	Exact and simulated outcomes; delayed and judged only under pruning or merging
[Auto × Human]	A program keeps it; a person lets it climb	Production systems with a reviewer on promotion, and CASCADE	Observed outcomes and judgments
[Auto × None]	A program keeps it; nothing lets it climb	Letta's ungated push; systems that promote nothing	As on the local gate
[Human × None]	A person keeps it; nothing lets it climb	Curation	Judged
[None × Auto]	No failure triggers a candidate; a threshold gates release	Prompts-as-code platforms and governance registries	No failure-driven loop
[None × Human]	No failure triggers a candidate; a person admits	Open standards, MCP registries, community hubs	No failure-driven loop
[None × None]	Nothing checks or promotes the edit	SkillRL and SkillForge, unchecked skill loops	Outcomes and judgments

Table 6: The gate grid read cell by cell: who keeps a local edit and who lets it climb, which systems sit in the cell, and the feedback signal they gate on. [Human × Auto] and [Human × Human] hold no surveyed system.

Failure	What goes wrong	The loop piece that answers it	Dominant class
Scaffold bloat	Rules and skills accumulate until the model follows them worse and the gate has no clean signal	Complexity penalty Ω in the gate; pruning and expiry as first-class gated operations	C1, C3
Poisoned memory	An attacker writes an entry that later retrieval trusts	Provenance on every entry; write gating; a rollback envelope	C2
Injection via tools and MCP	A tool's output or a connected server carries instructions into context	Narrow tool permissions; a privilege hierarchy over instructions; tool bindings pass the same promotion gate as any other coordinate	C2
Evaluation fragility	The suite that gates edits is itself edited, gamed, and read non-independently	Held-out evaluations scoped within evidence radius, weighted toward recurring modes, re-read as the target drifts, checked against attribution	C3 (primary), C1
Coordination failures	Multi-agent roles blur and context is lost between agents	Orchestration topology treated as a scaffold coordinate under the same gate and rollback discipline	C2
Patch overfitting	A scaffold tuned to one deployment degrades another	Evidence-radius discipline at promotion; receiving-side held-out evaluation	C2
Plasticity loss	Old rules, polluted memory, and drifting tool versions crowd out adaptation	Expiry and rollback as explicit loop operations rather than retrospective hygiene	C4

Table 7: The failure surface of patchwork scaffold maintenance: what goes wrong, the piece of the loop that answers it, and the feedback-signal class in which it dominates.

Eleven surveyed skill systems now treat removal as an operation that must pass a check, just as admission does (Zhang et al., 2026a; Wei et al., 2026; Pu et al., 2026; Wang et al., 2026d) (the supplementary material).

Persistent memory creates another route to lasting error. PoisonedRAG demonstrates attacks on retrieved knowledge (Zou et al., 2024), AgentPoison plants backdoors in long-term memory or knowledge bases (Chen et al., 2024), and memory control-flow attacks operate across major agent frameworks (Xu et al., 2026d). Tool outputs and connected servers can also insert instructions into the model's context (Greshake et al., 2023; Debenedetti et al., 2024). In CVE-2025-54136, a previously approved MCP configuration could be changed without renewed approval (Check Point Research, 2025).

These failures motivate provenance, write gating, and reversible memory changes (Su et al., 2026; Yu et al., 2026a), together with restricted tool permissions and an instruction privilege hierarchy (Wallace et al., 2024). The tool binding itself must be subject to controlled revision. Checking only the initial registration leaves later changes outside the decision that authorized it.

Rollback also needs to account for derivation. SkillSmith and SkillAudit support edit-level rollback (Wei et al., 2026; Gao et al., 2026a), and EvoUndo rejects edits that cannot be undone (Sah et al., 2026). If a contaminated skill has been used

to create other skills, however, restoring its earlier version does not remove those descendants: VaG finds that reversing a contaminated skill recovers little once other skills have been derived from it (Shang et al., 2026). This limited recovery supports checking contamination before commitment and tracking consequences after it. A version history is a prerequisite for this work, but it does not perform the recovery by itself.

E.2 History is not correction

Memory products have made substantial progress on a different part of maintenance: recording an entry's history. Letta's Context Repositories use git, Mem0 retains per-memory history, and Zep represents facts bitemporally (Letta, 2026; Mem0, 2026; Zep, 2026). These records can explain what the system remembers and how that state arose.

History does not guarantee correction. A study of five production backends finds that all five still return facts that were revoked (Shen et al., 2026a). The record contains the change, but retrieval does not reliably enforce it. Research systems likewise divide between provenance and write controls (Wang, 2026; Xu et al., 2026a; Margalit et al., 2026; Sun et al., 2026) and verifier-gated failure correction (Yu et al., 2026b; Wang et al., 2026a). The governed collaborative memory of Cuadros et al. (2026) combines provenance, versioning, and correction, earning the top memory score.

E.3 When the evaluator changes

Consider a pitch-review assistant that consistently overrates decks claiming a large market without supporting numbers. The team revises its rubric from "evaluate market size" to "require a numeric estimate and a source." The edit may improve reviews, but it also changes the standard used to evaluate subsequent behavior. If the same rubric judges future rubric revisions, the improvement process is modifying its own measuring instrument.

This is the reflexive-gate problem introduced with C3 feedback (Appendix C). The survey finds three scaffold systems reporting calibration of a judge against people: SkillForge, SkillChain, and SkillCoach (Liu et al., 2026b; Hu et al., 2026b; Zhu et al., 2026b). None reports calibration for a judge that is itself being edited. The missing result is therefore more specific than an absence of calibrated judges. It concerns maintaining calibration while the criterion changes.

Weight-level reinforcement learning has encountered a related problem in extending verifiable rewards to open-ended tasks. Some methods hold the judge fixed, using checklists, pairwise preferences, references, or rubrics (Gunjal et al., 2025; Jia et al., 2025; Shi et al., 2026; Jin et al., 2026). Others update the reward, rubric, or judge alongside the policy (Yu et al., 2025; Xu et al., 2026c; Ding et al., 2026a; Zhu et al., 2026a; Li et al., 2026a). Among the co-evolving methods reviewed, only RLPR reports agreement with human labels, and its reward is a probability rather than a rubric. The comparison is tabulated in the audit notes of the supplementary material; these weight-side methods are not included in the scaffold scores.

Even a fixed evaluator can become unreliable through repeated use. Optimizing a proxy beyond a point can reduce the underlying quality (Gao et al., 2022). Unequal exposure to evaluation data can create apparent performance gains (Singh et al., 2025). Evaluations of agent trajectories are also path-dependent, and deployment tasks drift while the suite remains fixed. Human criteria themselves can shift during grading (Shankar et al., 2024). Consequently, a gate needs its own held-out checks, calibration where judgment is involved, and revision history.

The failure-mode hypothesis discussed in Section 9 adds a conditional implication: if deployment failures concentrate in recurring modes, uniformly sampled tasks may underrepresent the modes that most need correction. Evaluation should then include those modes while preserving a held-out assessment. The requirement is broader than improving a benchmark score: score alone cannot tell a harness optimizer from trial and error (Ong et al., 2026), so the evaluation must also test the claimed link between failures and edits.

E.4 Scope, coordination, and drift

Some failures arise from the structure of the deployment. A study of multi-agent systems identifies fourteen failure modes, prominently including role confusion and context lost between agents (Cemri et al., 2025). Treating routing and role assignments as editable scaffold coordinates makes them subject to evaluation and rollback alongside instructions and skills.

Specialization creates a related boundary. A scaffold adapted to one deployment can degrade another because the tasks and preferred behavior differ. Receiving-side evaluation limits this risk

at promotion; the evidence radius records where the improvement has been tested. This does not eliminate uncertainty outside the evaluated cases.

Finally, the scaffold can become less adaptable as old rules, polluted memories, and drifting tool versions accumulate. Long-context degradation (Liu et al., 2023) provides one related mechanism. Expiry, pruning, and rollback must remain available throughout the system's lifetime, especially when delayed feedback arrives too slowly to keep the artifacts current.

The feedback classes help organize these risks without making them exclusive to one class. Exact checks invite suite overfitting and cheap accumulation; observable outcomes depend on channels vulnerable to poisoning and injection; judged feedback is exposed to proxy optimization and moving criteria; delayed feedback makes drift difficult to correct promptly. Production systems provide individual controls, but the surface stays largely unmitigated because no surveyed system runs them together.

F Governed sharing across organizations

Section 6 introduces receiving-side evaluation, and Section 9 states in one paragraph that no surveyed system holds all four properties of governed promotion across organizations. This appendix gives the receiving-side evidence that exists within one operator, the four-property audit with the closest approaches, the architecture that would connect them, the comparison with federated learning, and how far automation should go in each feedback class.

F.1 Receiving-side evaluation within one operator

The nesting of scopes (Fig. 4) makes an important limit visible. A lower average loss at a broad scope can conceal harm to an individual recipient. The architecture below therefore includes a held-out check at the receiving tenant. Versioning and access control govern whether an edit may be delivered and reversed; receiving-side evaluation tests whether it should be used there.

Several systems already evaluate transfers within one operator. ShareMem consolidates memories from one user for another and evaluates them in the receiving context (Hu et al., 2026a). XRepoSkill verifies a rule in at least two repositories (Guo et al., 2026b). Evo2Team promotes routing skills between teams and confirms them at the target (Wang and Cui, 2026). These are concrete instances of evidence being gathered beyond the source deployment.

Federated systems place the check elsewhere. EvolveNet merges a shared harness only when the merge fixes at least as many held-out items as it breaks, but that check occurs at the aggregator (Nie et al., 2026). FederatedSkill's server agent applies, adapts, or rejects each client's edit and records the decision (Yang et al., 2026a). Their clients are partitions or agent instances under one operator, not independently governed customer organizations. They do not instantiate a receiving-organization gate.

The location of the evaluation matters, and its coverage still matters after it is correctly located. ShareMem, XRepoSkill, and Evo2Team check the receiving context; EvolveNet checks the shared merge; FederatedSkill records a server decision. Even in Evo2Team, nearly a fifth of confirmed transfers fail on graphs not covered by confirmation. A receiving-side check provides necessary evidence for adoption, but finite evaluation does not guarantee performance on all future tasks.

F.2 Four properties and the closest approaches

The evidence identifies a practical integration problem. Local optimizers can propose and test changes. Release platforms can version artifacts and reverse deployments. Registries can govern admission and access. The surveyed systems do not connect these capabilities into a process that learns in one organization and shares the resulting improvement with another under equivalent controls. We audit that missing combination through four properties:

(a) **An auto-gated local loop:** an algorithmic acceptance criterion tests a proposed artifact change.
(b) **Cross-organization promotion:** artifacts can be promoted between independently governed organizations or customers.
(c) **Versioned lineage with access control:** a promoted artifact carries its history and audit trail, with policies controlling who may access it.
(d) **Rollback:** a harmful promotion can be reversed without redeploying the whole system.

The terms *cross-tenant* and *cross-organization* refer to the same boundary here. Sharing among

agents, users, or benchmark partitions within one operator does not establish property (b).

Table 8 compares the closest systems. FederatedSkill combines local adaptation with semantic skill exchange and an auditable server decision log (Yang et al., 2026a). EvolveNet evaluates shared merges and can return to the previous shared harness (Nie et al., 2026). Both remain within one operator. Governance registries supply lineage and rollback without a local learning loop, and MCP control planes supply access control alone. Rollback exists inside one federated round and inside single deployments; no system rolls back a promotion that has crossed an organization boundary, because none makes such a promotion.

F.3 An architecture that would close the gap

Receiving-side evaluation is an additional requirement on the promotion decision. It is kept separate from the four-property audit, which records loop, scope, and governance capabilities. A system could satisfy those capabilities while still making an unsupported decision about whether an artifact benefits its recipient.

An architecture that addresses both would contain five connected parts: per-tenant artifact pools with provenance; an aggregation or selection process that respects tenant isolation; held-out evaluation at the receiving tenant; versioned lineage and access control; and a defined reversal path for each promoted artifact. The recipient's check would reject a promotion that does not lower its own patch loss. VaG's result (Appendix E) suggests that the rollback envelope would also need to track an artifact's descendants.

F.4 Comparison with federated learning

Federated learning offers a useful comparison, but the representation being shared is different. FedAvg combines comparable numeric updates, and its privacy and deployment problems have been extensively studied (McMahan et al., 2017, 2018; Kairouz et al., 2019). Federated prompt tuning moves the learned object to a shared prompt (Guo et al., 2022). The nearest scaffold systems instead select, revise, or merge symbolic edits, often through a language model. Averaging is an appropriate analogy only where the artifact updates have a mergeable representation.

F.5 Automation by feedback class

The extent of automation should depend on the feedback. Exact checks provide the most direct setting for testing a transferred skill, although the recipient still needs relevant test cases. Observed outcomes require evaluation on the recipient's workload because the sender's outcome is not portable evidence of success. Where quality is judged, a human promotion gate is a design feature, supported by lineage and rollback. Delayed outcomes cannot support the per-edit automatic local gate in property (a); those systems still benefit from provenance and reversibility around proposals and milestones.

Thus the four properties define a target for the settings that can support an automatic local gate. They should not be interpreted as a requirement to automate every decision in every domain. The common requirement is that the evidence attached to an edit remain meaningful where it is used, and that the system preserve the ability to inspect and reverse its adoption.

G Research lines

Section 8 names five research lines in one paragraph, each of which built one part of the loop. This appendix gives the table of what each line started from, built, and left open, the citations behind each line, the failure records that none of the lines set out to build, the attribution work concentrated in harness optimization, and what the convergence of the lines does and does not show.

The uneven architecture has a history. Different research communities began with different problems: improving prompts, learning in context, selecting agent workflows, maintaining skill libraries, or transferring experience. Each developed the part of the loop most relevant to its initial problem. Table 9 places five such lines side by side.

Prompt optimization and reflective evolution developed ways to propose edits from trajectory evidence, together with an argument for adapting artifacts on a faster schedule than weights (Agrawal et al., 2025; Tiwari et al., 2026). Reflective Context Learning formalizes learning in context and the need to control generalization, corresponding to the complexity penalty in the acceptance rule (Vassilyev et al., 2026). Harness optimization treats source-level orchestration changes as the learned object and repeatedly identifies attribution as its difficult step (Lee et al., 2026; Lin et al., 2026a; Cai et al., 2026; Zehle, 2026; Nguyen et al., 2026;

System	(a) auto-gated local loop	(b) cross-organization promotion	(c) versioned lineage with access control	(d) rollback
FederatedSkill (Yang et al., 2026a)	full	partial: privacy-preserving aggregation of skill edits across heterogeneous clients that sit under one operator	partial: auditable per-modification decision log	none
EvolveNet (Nie et al., 2026)	full	partial: held-out merge gate at the aggregator, but clients are benchmark partitions, not organizations	none: no lineage, no access control	full: rollback to the previous shared harness after a failed round
SkillForge (Liu et al., 2026b)	none: a calibrated judge selects the failures, nothing tests the rewrite	none: skill versions stay in the scenario deployment	partial	not documented
Governance-first registries (Kang and Wang, 2026; Qin et al., 2026; Quessada-Vial, 2026)	none: no inner loop	none	partial: lineage without access control	full, with evidence
MCP control planes (agentic-community, 2026)	none: no inner loop	none	partial: access control without lineage	not reported

Table 8: The closest approaches to governed cross-organization scaffold optimization, scored against the four properties.

Chen et al., 2026; Li et al., 2026c).

The lifecycle literature starts from a different constraint: skill pools can become too large or internally inconsistent to remain useful. Its contributions include evaluated retirement, hygiene, and optimization under platform limits (Zhang et al., 2026b; Tanjim et al., 2026; Liu et al., 2026a; Zhang et al., 2026a; Wei et al., 2026; Pu et al., 2026; Wang et al., 2026d). Transfer and federation ask how far a learned artifact remains useful, progressing from cross-task measurements to receiving-team confirmation and shared client updates (Shalev et al., 2026; Li et al., 2026b; Qu et al., 2026; Ling et al., 2026; Wang and Cui, 2026; Yang et al., 2026a; Nie et al., 2026).

G.1 Failure records

A useful failure history must retain what went wrong in a form that supports later comparison. Production memory more often records preferences, successes, or conversation history. Only one field report in the corpus describes a store of dead ends intended to prevent their recurrence, and that evidence comes from a single self-reported deployment (Helwig, 2026). The persistent store may therefore be extensive while the particular record needed for failure-driven adaptation is missing.

Even when failures are retained, nearly every system uses them in arrival order. Few explicitly choose which recurring modes deserve attention. Task-CoEvolve's variance-weighted task sampling and AutoSaddler's mini-batch failure signals are exceptions (Miyai et al., 2026; Park et al., 2026). They weight the experience used for the next update rather than letting the most recent session determine it. The general loop of Section 2 would prioritize both recurrence and severity; this remains uncommon in the surveyed implementations.

G.2 Attribution as the shared bottleneck

A failed output does not reveal whether its cause was an instruction, a retrieved memory, a skill, or the route through a group of agents. Harness-optimization papers repeatedly identify this attribution problem as the bottleneck (Lin et al., 2026a; Zehle, 2026; Lee et al., 2026; Nguyen et al., 2026; Liu et al., 2026a). An evaluation of harness optimizers reaches the same conclusion: by downstream score alone, a harness optimizer cannot be told from trial and error (Ong et al., 2026).

Existing solutions usually operate within one substrate. Meta-Harness preserves candidate harnesses and the record of which edits helped (Lee et al., 2026). SkillsVote attributes outcomes to the skill, exploration, or environment before admitting an update (Liu et al., 2026a). SkillOpt-Lite keeps only the attributes that recur across rollouts, and its ablation finds this consensus mechanism load-bearing (Shen et al., 2026b). These methods give the proposal process more useful evidence. None

Research line	Started from	The part of the loop it built	What it leaves open
Reflective evolution	Prompt optimization and continual learning	The candidate edit: proposal by reflection over trajectory evidence; the two-timescale case for editing outside the weights	A persistent artifact; a bounded step; a held-out gate
Reflective context	Learning in context as a setting of its own	The acceptance gate's penalty: the learned object in context space; a generalization penalty, the Ω of the gate	A cross-deployment promotion loop
Harness optimization	Orchestration and harness engineering	Credit assignment, the trace: attribution of a failure to one slot of the orchestration graph; source-level edits	Attribution across substrates; a governed promotion loop; access control; rollback
Lifecycle and multi-objective	Skill pools that bloat under platform limits	The gate on removal: gated retirement; hygiene as a precondition; a hard frontier as the shape of Ω	Closing the loop as descent; retiring a promoted artifact's descendants
Transfer and federation	How far an edit travels	The promotion loop, the check on arrival: evidence radius measured one task at a time; cross-client aggregation; receiving-side confirmation	A cross-organization gate, lineage, and rollback

Table 9: Five research lines converging on artifact-layer descent: the problem each started from, the part of the loop it built, and what it leaves open.

solves the general attribution problem across all six scaffold substrates.

G.3 What the convergence shows

This history helps explain why evaluation and proposal mechanisms are more developed than failure capture. Each research line inherits its loss from its benchmark and begins with available trajectories. Building a record of recurring failures was none of their starting problems. Attribution receives substantial attention, but usually within the substrate a particular line studies. The local loop has consequently developed from evaluation backward toward the evidence needed to choose a useful edit.

The convergence supports a shared decomposition of the system into weights and editable artifacts. GEPA, Meta-Harness, MOSS, RCL, PACE, and the harness-optimization papers all use it. A system that updates both, SIA, still treats them as separate levers (Hebbar et al., 2026). The convergence does not establish that one architecture is optimal, and none of these lines documents the complete cross-organization process. It shows that different starting problems repeatedly lead to comparable mechanisms for proposing, testing, retaining, and reusing changes.

H Neighboring surveys and the prompt-optimization lineage

Section 8 answers the originality question in three paragraphs. This appendix records, survey by survey, what each neighbor organizes by and where this survey departs from it, then the two closest governance precedents and the prompt-optimization lineage.

Existing surveys organize the field around agent evolution, persistent state, context engineering, skills, memory security, and harness design. This survey follows a different unit of analysis: the decision that accepts a persistent change, the feedback supporting that decision, and the scope in which the change is subsequently used. Table 10 records the distinction for each neighboring survey.

The closest governance precedents are the survey of persistent-state agents by Ding et al. (2026c) and the memory-security survey by Lin et al. (2026b). The former finds greater attention to accumulating and retrieving state than to governing, recovering, or relinquishing it. The latter treats scoping and rollback as security primitives. Neither asks who gates an edit, and neither covers promotion across organizations.

Prompt optimization supplies another important lineage: OPRO, EvoPrompt, Promptbreeder, DSPy, and TextGrad optimize a prompt or program once against development evidence and are not deployed loops (Yang et al., 2024; Guo et al., 2024; Fernando et al., 2023; Khattab et al., 2023; Yuksekgonul et al., 2024). ACE persists an evolving playbook but never promotes it (Zhang et al., 2025b). Agent symbolic learning treats prompts, tools, and pipelines as symbolic parameters (Zhou et al., 2024); STOP improves its own scaffolding program (Zelikman et al., 2024); and SICA edits its harness to improve benchmark performance (Robeyns et al., 2025). These works establish many of the underlying mechanisms.

Survey	Scope	How this survey differs
Gao et al. (2026b)	self-evolving agents	organizes by what, when, how, and where an agent evolves; the gate that accepts an edit, and the edit's scope, are mentioned but not classified
Fang et al. (2025)	self-evolving agents	organizes by inputs, agent system, environment, and optimizer; loop closure appears in passing, gates and promotion do not
Zhou et al. (2026)	self-evolving coding agents (108 methods)	organizes by evolution target, timing, and code signal; closest on reversibility, with no gate or promotion axis
Ding et al. (2026c)	persistent-state agents (435 works)	organizes by authority, scope, mutability, provenance, and recoverability of state; governance is an axis, the gate on an edit is not
Lin et al. (2026b)	memory security (∼90 works)	organizes by lifecycle phase and security objective; principal scoping and rollback are axes, feedback and promotion are not
Guo et al. (2026a)	agent system and harness design	organizes by six runtime responsibilities; touches all four questions without classifying by any
Xia et al. (2026a)	skill-mediated agents (8 systems)	organizes by four responsibility layers and ten patterns; loop closure and governance appear as patterns, not as axes
Ding et al. (2026b)	skill evaluation and evolution	organizes by four evolution paradigms and six benchmark classes; evaluation is the subject, the gate it feeds is not
Mei et al. (2025)	context engineering (1,411 papers)	organizes by components and system implementations; adaptation loops lie outside its scope

Table 10: Neighboring surveys and where this one departs from each. This survey classifies by loop closure, gate type, signal class, and governance, and is the only one to cover promotion across organizations.

The contribution here is their joint reading at deployment scope: whether an artifact persists, whether a local acceptance decision is made, which signal supports it, and what authorizes use elsewhere. The 256-system corpus, two-loop classification, feedback classes, and governance audit make that comparison explicit. An extended discussion in the supplementary material separates these contributions from their precedents.

I Why specialization lives outside the weights

Section 9 gives the practical and the structural answer in two paragraphs. This appendix gives both in full: the different requirements of a general model and a deployment, the evidence from preference training and adaptation cost, the adapters that could absorb local requirements, and the two prior analyses of error with their evidence and their limits.

A general model and a particular deployment face different requirements. The model must work across many settings. A deployment must follow its own conventions, honor its users' preferences, and avoid failures that may be rare globally but frequent locally. The scaffold provides a place to encode those requirements with a scope narrower than the model's entire user population.

Work on reinforcement learning from human feedback reports reduced output diversity and more homogeneous preferences (Kirk et al., 2024; Padmakumar and He, 2023). This helps motivate the distinction between broad model behavior and customer-specific requirements. Tiwari et al. (2026) approach it from adaptation cost: in-context changes are cheap but capacity-bounded, while in-weight adaptation is more expressive and can introduce forgetting and loss of plasticity (Dohare et al., 2024).

Per-customer adapters (Hu et al., 2021), distilled models, and learned routing can absorb some local requirements. They also introduce separate training, evaluation, and serving lifecycles. In practice they are rare. Adaptation lands in persistent artifacts, the one substrate that is cheap, inspectable, reversible, scoped to one customer, and updatable daily.

Two prior analyses offer a hypothesis for why targeted artifact changes may be effective. Arbuzov et al. (2025) argue that long-output reliability depends on a sparse set of key decisions rather than a uniform error probability at every token. Their evidence includes long-range dependence measured on one natural-text corpus, where fewer than a tenth of tokens depend on distant context, and the accuracy recovered through majority voting on reasoning benchmarks. The argument is limited when the number of consequential decisions grows with the task, as in long-horizon agentic work.

Arbuzov et al. (2026) consider a bounded deployment with recurring tasks, tools, and evaluator expectations. They argue that failures there fall into a catalogue of modes whose size grows slowly with observed experience. Under their assumptions,

the intervention budget grows polylogarithmically with sequence length and becomes constant once the catalogue saturates. Their negative result is equally relevant: a finite catalogue cannot cover an unbounded domain.

If these hypotheses generalize, a deployment may be improved through a manageable set of targeted interventions. Persistent rules, memories, skills, and tool changes provide an inspectable representation for those interventions. On that hypothesis the gated and scoped artifact layer is not a workaround for an inadequate model but the architecture the error structure calls for: a frontier model that generalizes, a local scaffold that specializes, and reliability that comes from governing the specialization. The survey does not test the hypotheses; its findings about acceptance gates, feedback, and promotion follow from the corpus independently of them.